

WebCompass: Towards Multimodal Web Coding Evaluation for Code Language Models

Xinping Lei[†], Xinyu Che[†], Junqi Xiong[†], Chenchen Zhang[†], Yukai Huang[†], Chenyu Zhou[†], Haoyang Huang, Minghao Liu, Letian Zhu, Hongyi Ye, Jinhua Hao, Ken Deng, Zizheng Zhan, Han Li, Dailin Li, Yifan Yao, Ming Sun, Zhaoxiang Zhang, Jiaheng Liu^{*}

Nanjing University Kuaishou Technology

[†]Equal contribution. ^{*}Corresponding author.

Abstract

Large language models are rapidly evolving into interactive coding agents capable of end-to-end web coding, yet existing benchmarks evaluate only narrow slices of this capability—typically text-conditioned generation with static-correctness metrics—leaving visual fidelity, interaction quality, and codebase-level reasoning largely unmeasured. We introduce **WebCompass**, a comprehensive, multimodal benchmark that provides a *unified lifecycle evaluation* of web engineering capability. Recognizing that real-world web coding is an iterative cycle of generation, editing, and repair, WebCompass spans three input modalities (**text, image, and video**) and three tightly coupled task types (**generation, editing, and repair**), yielding seven complementary task categories that closely mirror professional workflows. Through a multi-stage, human-in-the-loop pipeline, we curate high-quality instances covering 15 generation domains, 16 editing operation types, and 11 repair defect types, each annotated at Easy/Medium/Hard difficulty levels. On the evaluation side, we adopt a *checklist-guided LLM-as-a-Judge* protocol for editing and repair, and propose a novel **Agent-as-a-Judge** paradigm for generation that autonomously executes generated websites in a real browser, explores interactive behaviors via the Model Context Protocol (MCP), and iteratively synthesizes targeted test cases—closely approximating human acceptance testing. We evaluate a diverse set of representative closed-source and open-source models and observe that: (1) closed-source models remain substantially stronger and more balanced; (2) editing and repair exhibit distinct difficulty profiles, with repair preserving interactivity better but remaining execution-challenging; (3) aesthetics is the most persistent bottleneck, especially for open-source models; and (4) framework choice materially affects outcomes, with Vue consistently challenging while React and Vanilla/HTML perform more strongly depending on task type. All benchmark data¹, evaluation code², and supplementary artifacts³ are publicly available.

1 Introduction

Large Language Models (LLMs) have rapidly evolved from passive code assistants into interactive coding agents capable of implementing substantial software changes from natural-language instructions (Yang et al., 2024b; Wang et al., 2024; Cognition AI, 2024). This progress is especially evident in **web development**, where outputs can be directly executed, visually inspected, and iteratively refined. A growing body of work has proposed benchmarks that span different task types and input modalities for web coding (Table 1).

Yet evaluating web coding is fundamentally different from evaluating traditional code generation. Success depends not only on functional correctness, but also on visual fidelity, interaction behavior, responsiveness, accessibility, and overall user experience. These aspects are difficult to capture with standard code-centric metrics such as pass@k on HumanEval (Chen et al., 2021) or unit-test pass rates on SWE-Bench (Jimenez et al., 2023), which focus on algorithmic correctness or repository-level bug fixing rather than interactive front-end applications.

To address this gap, we introduce **WebCompass**, a unified multimodal benchmark and evaluation framework for web coding. WebCompass spans **text, image, and video** inputs, covers **generation, editing, and repair** tasks, and adopts **task-aware evaluation** tailored to each setting. For editing and repair, we use a *checklist-guided LLM-as-a-Judge* protocol (Zheng et al., 2023), which is well suited to

¹<https://huggingface.co/datasets/NJU-LINK/WebCompass>

²<https://github.com/NJU-LINK/WebCompass>

³<https://nju-link.github.io/WebCompass/>

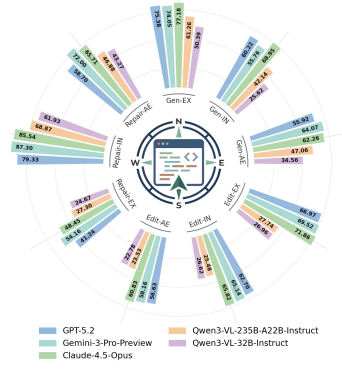


Figure 1: Radar chart of model performance across all seven task types in WebCompass.

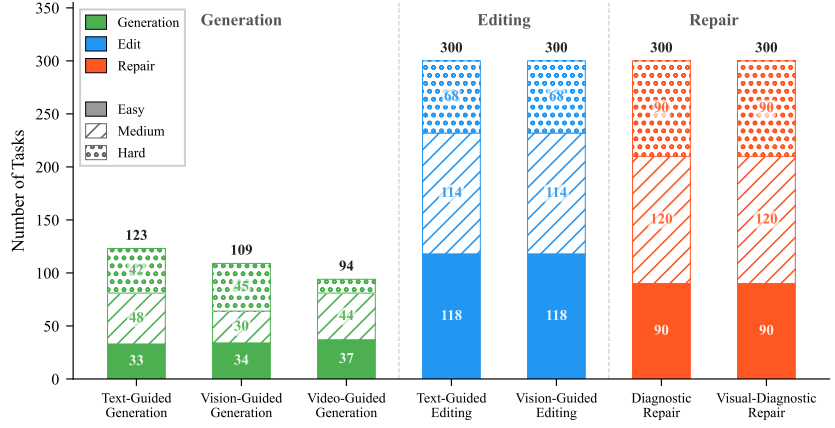


Figure 2: Difficulty distribution of WebCompass.

Table 1: Comparison with prior web coding benchmarks. WebCompass is the first to support all three task types across text, image, and video modalities. **Gen.**=Generation, **Edit**=number of supported editing categories, **Rep.**=number of supported repair categories, **Multi-page**=project-level multi-page testing, **Interact.**=interactive functionality evaluation, **Visual**=aesthetics and visual fidelity evaluation, **Agentic**=Agent-as-a-Judge dynamic testing (using LLM agents to interact with browsers and synthesize tests), **Reverse**=reverse-engineered deterministic repair tasks. A red cross indicates that the task family is not supported. Data sizes are reported as the number of tasks or question-answer pairs.

Benchmark	Size	Gen.	Edit (#)	Rep. (#)	Multi-page	Interact.	Visual	Agentic	Reverse	Input Modality
<i>Generation-Only Benchmarks</i>										
Interaction2Code (Wan et al., 2024)	504	✓	×	×	×	✓	✓	×	×	Image
FronTalk (Wu et al., 2025)	1000	✓	×	×	✓	✓	✓	×	×	Text Image
Web-Bench (?)	1000	✓	×	×	✓	✓	×	×	×	Text Image
FrontendBench (Zhu et al., 2025)	148	✓	×	×	×	✓	×	×	×	Text
WebApp1K (?)	1000	✓	×	×	✓	×	✓	×	×	Text
IWR-Bench (?)	113	✓	×	×	✓	✓	✓	×	×	Video
WebGen-Bench (Lu et al., 2025)	101	✓	×	×	✓	✓	×	×	×	Text
<i>Multi-Task Benchmarks</i>										
SWE-bench MM (Yang et al., 2024a)	517	×	3	4	✓	×	×	×	×	Text Image
DesignBench (Xiao et al., 2025)	900	✓	6	6	✓	×	✓	×	×	Image
WebCompass (Ours)	1526	✓	16	11	✓	✓	✓	✓	✓	Text Image Video

patch-based outputs with constrained solution spaces. For generation, we propose an **Agent-as-a-Judge** protocol (Zhuge et al., 2024), in which an autonomous agent launches the generated website in a real browser, explores it through MCP, synthesizes targeted test cases, and scores the result based on execution.

This design reflects the differing nature of web coding tasks. Editing and repair are localized and checklist-aligned, making diff-level inspection and before/after screenshots sufficient for reliable evaluation. Generation, by contrast, is open-ended and long-horizon, with correctness often depending on multi-step runtime behavior that static inspection cannot capture. By combining multimodal task coverage with execution-based evaluation, WebCompass provides a more realistic and scalable benchmark for assessing web coding agents.

Contributions. (1) **Unified lifecycle coverage.** Unlike prior benchmarks that target isolated tasks or modalities (Table 1), WebCompass jointly evaluates generation, editing, and repair across text, image, and video inputs, enabling cross-task and cross-modality comparisons within a single framework. (2) **Rigorous and deterministic task construction.** We refine underspecified queries into structured design documents for generation, synthesize context-consistent requirements without leaking implementation details for editing, and provide exact search/replace annotations mapping buggy code to clean targets for repair, ensuring reproducible evaluation. (3) **Task-aware evaluation paradigms.** We introduce an Agent-as-a-Judge protocol that combines real-browser interaction with iterative test-case synthesis for open-ended generation tasks, complementing checklist-guided LLM-as-a-Judge for constrained patch-based tasks.

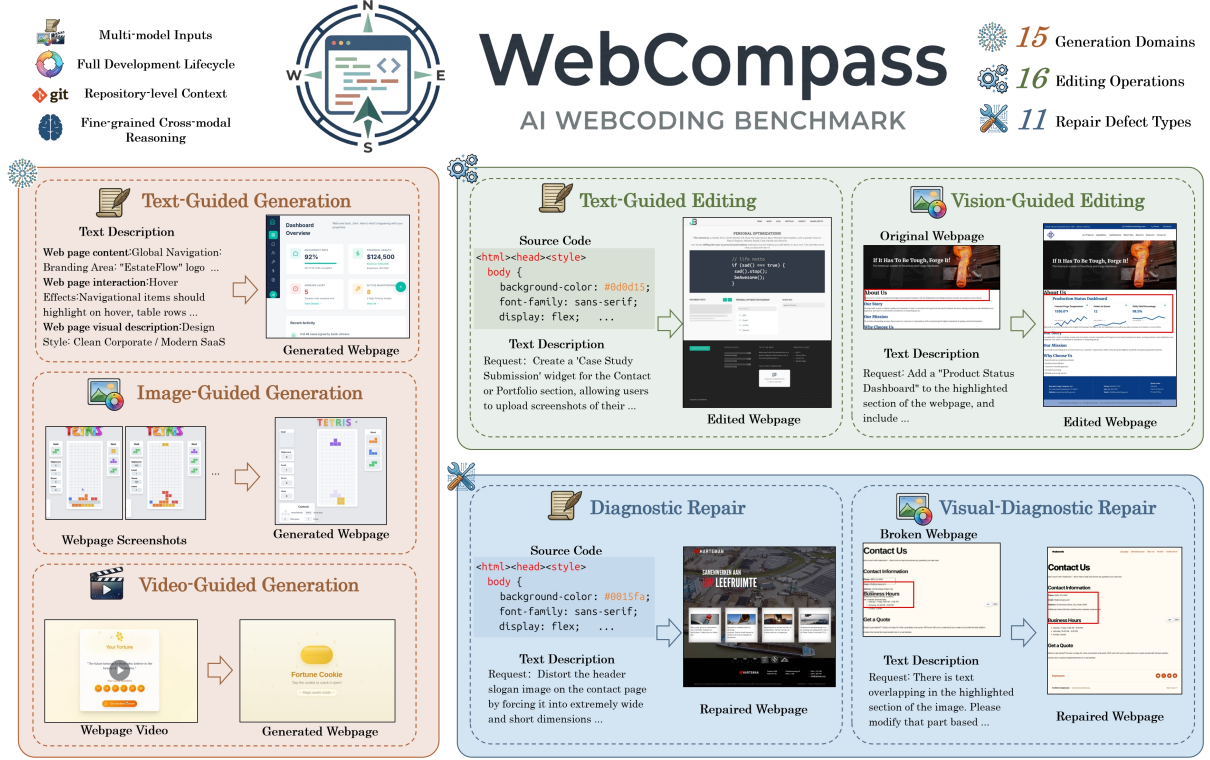


Figure 3: Overview of WebCompass. The benchmark supports three input modalities (text, image, video) and three task types (generation, editing, repair), resulting in seven complementary task categories that cover the full lifecycle of web development.

2 WebCompass

2.1 Overview

WebCompass supports three input modalities (text, image, and video) and three types of web coding tasks (generation, editing, and repair), resulting in seven task categories: *Text-Guided Generation* (text-conditioned web generation), *Vision-Guided Generation* (image-conditioned web generation), *Video-Guided Generation* (video-conditioned web generation), *Text-Guided Editing* (text-instructed web editing via patches), *Vision-Guided Editing* (image-grounded web editing via patches), *Diagnostic Repair* (text-described web repair via patches), and *Visual-Diagnostic Repair* (image-grounded web repair via patches). Each task is designed to closely reflect real-world development scenarios. We define each task as follows:

1. **Text-Guided Generation.** The input is a textual specification of a target web page, consisting of three aspects: (i) page content, (ii) interaction behaviors, and (iii) visual appearance. The model is required to output a complete web code repository that satisfies the specification.
2. **Vision-Guided Generation.** The input comprises multiple screenshots of a web page. Beyond presenting content, layout, and visual styling, the screenshots are also intended to capture interactive functionalities. Depending on the data source, we consider two types of screenshot sets: (i) a collection covering the main page and its subpages, and (ii) a sequence capturing page state changes during browsing. The model is required to reproduce a web code repository whose visual appearance and functionality match the screenshots.
3. **Video-Guided Generation.** The input is a screen-recorded browsing video containing multiple user interactions. The model is required to generate a web code repository whose appearance and functionality are consistent with those demonstrated in the video.
4. **Text-Guided Editing.** The input includes a web code repository and a text-based editing instruction. The model is required to output a *code patch* that edits the repository such that the updated web page meets the instructions.
5. **Vision-Guided Editing.** The input includes a screenshot of the current web page, the corresponding web code repository, and an editing instruction. The model is required to output a *code patch* that modifies the repository so that the edited web page satisfies the instruction.

-
6. **Diagnostic Repair.** The input includes a web code repository and a textual description of the existing issues. The model is required to output a *code patch* that repairs the repository and resolves the described problems.
 7. **Visual-Diagnostic Repair.** The input includes a screenshot of the current web page, the web code repository, and a description of the existing issues. The model is required to output a *code patch* that repairs the repository and resolves the described problems.

Taken together, WebCompass serves as a comprehensive benchmark to evaluate the capabilities of multimodal models in realistic web engineering scenarios. Beyond basic code generation, it rigorously assesses a model’s proficiency across several critical dimensions: (1) **Nuanced User Intent Understanding**, encompassing layout structure, aesthetic design styles, and complex interaction logic; (2) **Fine-grained Cross-modal Reasoning**, requiring precise alignment between visual inputs (images/videos) and code implementations; (3) **Repository-level Context Awareness**, testing the ability to maintain consistency within existing codebases during editing and repairing; and (4) **Diagnostic & Problem-Solving Skills**, specifically for identifying and fixing semantic or visual anomalies.

2.2 Data Collection

To ensure the benchmark reflects real-world scenarios, we employ a multi-stage, human-in-the-loop pipeline to construct a high-quality benchmark covering all seven task types. Figure 4 illustrates the overall process.

2.2.1 Text-Guided Generation.

We design the Text-Guided Generation set to (i) contain realistic and actionable requirements and (ii) cover diverse web page types. We therefore collect initial queries from multiple complementary sources: WebGen-Bench (Lu et al., 2025) (manually constructed queries), ArtifactsBench (Zhang et al., 2025) (diverse page categories with rigorous filtering), BigCode Arena (real user requests), and high-quality web showcases from V0 (an AI IDE for web coding). These sources form our initial query pool. To reduce redundancy, we embed queries using BGE-M3 and perform k -means clustering to obtain a deduplicated candidate set. We then use an LLM to assign category and difficulty labels to each query (five independent annotations per query), taking the majority vote as the final label. Finally, we perform stratified sampling across categories and difficulties to obtain 123 text-guided generation queries.

However, we observe that queries from everyday usage scenarios are often *underspecified*, leading to large variations in generated pages across models. While such low-constraint queries can test a model’s creativity, they hinder automated evaluation because creativity and implicit-intent matching are subjective and difficult to judge automatically—it is unclear whether the model is being “overly clever” or truly aligned with user intent. To address this, we prompt an LLM to act as a product manager and elaborate each underspecified request into a structured web design document covering (1) page content, (2) interaction behaviors, and (3) visual appearance.

2.2.2 Vision-Guided Generation.

Although many existing datasets include webpage screenshots, most contain relatively simple UIs that are insufficient to challenge modern models. We observe that WebRenderBench provides a large number of visually complex webpages, but typically only includes a single screenshot per website. We thus perform data augmentation: we parse the subpage URLs referenced in `index.html`, randomly select two, and use Playwright to capture their screenshots. To further test whether models can reproduce multi-page websites and their dependency relationships, we inject a JavaScript overlay into the main-page screenshot to highlight the positions of subpage URLs with colored bounding boxes. Due to network instability and dynamic content loading, screenshots may contain artifacts. We therefore conduct multiple rounds of LLM-based verification as an initial filter, followed by manual inspection.

In addition, most existing datasets only provide *static* screenshots and lack dynamic webpage content. Although Interaction2Code (Wan et al., 2024) supplies multiple images to convey certain interaction information, it still cannot adequately represent animations and complex interaction patterns. To fill this gap, we browse diverse webpages from V0 and Figma and manually extract keyframes capturing critical state changes. These two components—augmented multi-page screenshots and dynamic keyframe sequences—together constitute the Vision-Guided Generation test set.

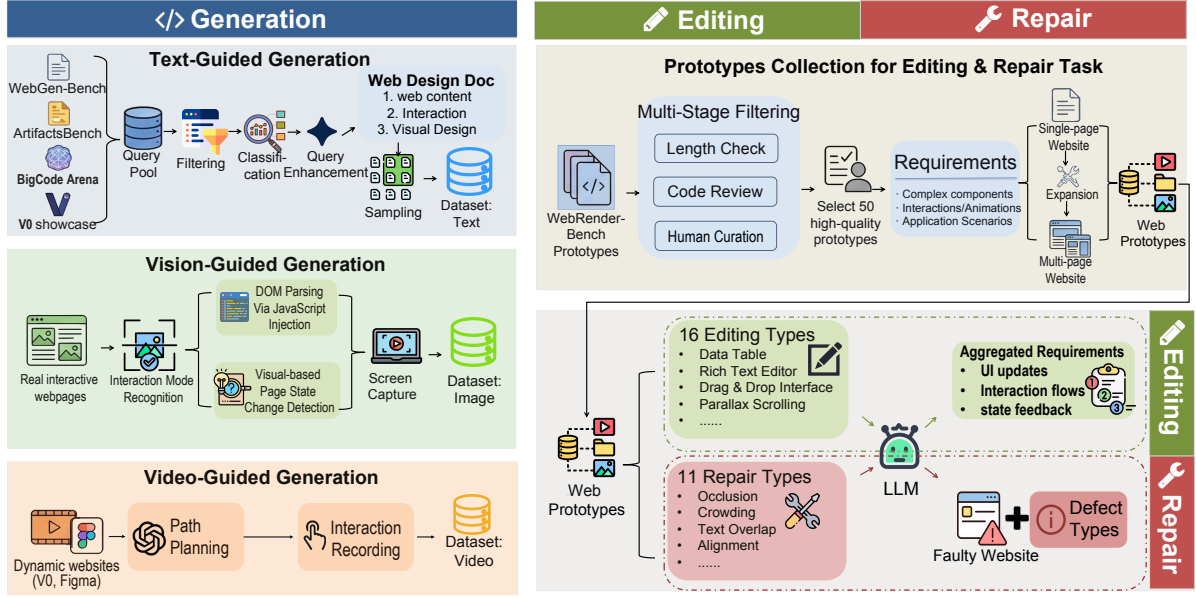


Figure 4: Data construction pipeline for WebCompass. **Top:** prototypes are collected through multi-stage filtering, manual selection, and page-level expansion. **Bottom:** each prototype is converted into editing tasks (left, green) or repair tasks (right, red) following task-type-specific procedures.

2.2.3 Video-Guided Generation.

Compared to text and images, videos can more clearly convey dynamic effects such as animations and multi-step interactions. To emphasize this advantage, we manually select webpages from V0 and Figma with rich dynamic behaviors across different categories, browse them, and record interaction videos. Annotators are instructed to first explore each webpage, plan a comprehensive exploration path, and then conduct the final recording to ensure thorough coverage of all interactive features.

2.2.4 Editing & Repair Task Data Collection Pipeline

Prototype Collection for Editing & Repair. Both editing and repair tasks share a common pool of high-quality web prototypes (Figure 4, top). We construct these prototypes from the WebRenderBench test set via a three-stage pipeline: *length filtering* $\rightarrow$ *automatic quality scoring* $\rightarrow$ *human curation*, then expand each selected prototype into single-page and multi-page variants.

- **Stage 1: Length filtering.** We constrain the total character count across all code files to 32k–64k, with each individual file no longer than 48k characters. These bounds approximate the multi-file coordination complexity of medium-to-large front-end projects, while avoiding overly small instances (lacking difficulty) or overly large ones (inducing context truncation and unstable evaluation).
- **Stage 2: Quality scoring.** For candidates satisfying the length constraints, we use GPT-4o to perform a code review on a 10-point scale and retain those scoring ≥ 9 , yielding 81 candidates.
- **Stage 3: Human curation and expansion.** We manually select 50 high-quality prototypes. Each prototype is kept as a SINGLE-PAGE website and additionally extended into a MULTI-PAGE website by adding extra pages, inter-page navigation, and shared resources. Together, the two variants constitute the *Web Prototypes* used for all downstream task construction.

Text-Guided and Vision-Guided Editing. Starting from each web prototype as the executable *source* website, we create editing instances by introducing new or enhanced requirements aligned with 16 predefined high-level task types covering complex components (e.g., data tables, rich-text editors, drag-and-drop interfaces), interaction/animation effects (e.g., parallax scrolling), and holistic application scenarios (Figure 4, bottom-left). For every task type, we aggregate requirements that specify *what* to change—including UI updates, interaction flows, and state feedback—while deliberately omitting implementation details (e.g., class names, selectors, or CSS values) to ensure fairness and realism. The resulting requirements, paired with the source website, form the editing instances; *Vision-Guided* variants additionally supply a reference screenshot in lieu of (or alongside) the textual instruction.

Diagnostic and Visual-Diagnostic Repair. Repair tasks are constructed in a verifiable *reverse* manner (Figure 4, bottom-right). We treat a clean web prototype as the *destination* and use an LLM to inject explainable, observable front-end defects drawn from 11 repair types, producing the faulty *source* website. The model is then required to repair the source back to the destination. The injected defects span three dimensions:

- **Visual layout:** occlusion, crowding, text overlap, misalignment, insufficient contrast, overflow, and distorted proportions.
- **Semantics & structure:** incorrect semantic/nesting structures and missing attributes.
- **Interaction usability:** broken interactions and loss of interactivity.

We then generate natural-language repair instructions that provide vague hints about potential defect types or underlying issues, rather than a complete description of the problem, ensuring no implementation details are leaked. To guarantee determinism and support automatic evaluation, each repair instance includes an exact text-level modification annotation (search/replace) that is the strict inverse of the defect-injection edits. This design ensures (i) a uniquely correct, runnable solution, (ii) reproducible transformation from source to destination, and (iii) automated verification and error localization. Throughout, we enforce contextual consistency and the “specify goals, not methods” principle.

Ecological validity of injected defects. The 11 defect categories are not arbitrarily chosen. They are the product of a systematic analysis of over 200 real-world community submissions on V0 and corresponding GitHub Issues, from which we identified the most frequently occurring front-end anti-patterns. Each category (e.g., Occlusion, Overflow, Loss of Interactivity) corresponds to a high-frequency failure mode observed in practice. By grounding our synthetic defects in this empirical distribution, we ensure ecological representativeness—models are tested on the kinds of bugs they are most likely to encounter in real-world web development, rather than on artificial corner cases.

2.3 Quality Control

We apply a multi-layered quality assurance process across all task types:

Automated checks. Before human review, every instance passes through a suite of automated validators: (i) all code repositories must compile and render without fatal errors in a headless Chromium environment; (ii) editing and repair patches must apply cleanly to their respective source repositories; and (iii) repair search/replace annotations are verified to be the exact inverse of the defect-injection edits, guaranteeing a unique, deterministic solution.

LLM-assisted screening. We use an LLM to perform multi-round quality checks on generated requirements and screenshots. For Vision-Guided Generation, the LLM verifies that screenshots are complete (no blank regions, missing assets, or broken layouts caused by network issues). For edit and repair tasks, the LLM checks that natural-language instructions are unambiguous, do not leak implementation details, and are consistent with the underlying code changes.

Human curation. All instances undergo a final round of expert human review. Annotators verify (i) the correctness and completeness of task descriptions, (ii) the visual quality of screenshots and videos, (iii) the appropriateness of difficulty labels (Easy/Medium/Hard), and (iv) the alignment between requirements and ground-truth patches. Instances that fail any criterion are revised or discarded.

2.4 Dataset Statistics

We propose a fine-grained taxonomy for the generation, editing, and repair tasks, as detailed in Table 2. The **generation** task encompasses 15 distinct domains: “E-commerce & Fintech”, “Enterprise & Productivity”, “Social & Communication”, “Data Science & Analytics”, “Content Creation & Multimedia”, “Entertainment & Streaming”, “Game Development & Gaming”, “Education & Learning”, “Simulation & Scientific Modeling”, “Infrastructure & System Management”, “DevTools & Engineering”, “Logic & Workflow Visualization”, “Location Services & Transit”, “Information & Personal Branding”, and “Lifestyle & Niche Utilities”. The **editing** task consists of sixteen operation types: Data Table, Rich Text Editor, Drag & Drop Interface, Tree View, Real-time Dashboard, Infinite Scroll, Async Form Validation, File Upload with Progress, Parallax Scrolling, Page Transitions, Particle Effects, Skeleton Loading, Shopping Cart, User Authentication, Multi-step Wizard, and Notification Center. The **repair** task addresses eleven types of front-end defects spanning visual, semantic, and interactive dimensions: Occlusion, Crowding, Text Overlap, Alignment, Color & Contrast, Overflow, Sizing/Proportion, Loss of Interactivity, Semantic Error, Nesting Error, and Missing Attributes.

Table 2: Detailed taxonomy of Generation, Editing, and Repair tasks in WebCompass. Generation covers 15 application domains; Editing defines 16 modification operations; Repair addresses 11 front-end defect types spanning visual, semantic, and interactive dimensions.

Generation (15 Types)		Editing (16 Types)		Repair (11 Types)	
1	E-commerce & Fintech	1	Data Table	1	Occlusion
2	Enterprise & Productivity	2	Rich Text Editor	2	Crowding
3	Social & Communication	3	Drag & Drop Interface	3	Text Overlap
4	Data Science & Analytics	4	Tree View	4	Alignment
5	Content Creation & Multimedia	5	Real-time Dashboard	5	Color & Contrast
6	Entertainment & Streaming	6	Infinite Scroll	6	Overflow
7	Game Development & Gaming	7	Async Form Validation	7	Sizing/Proportion
8	Education & Learning	8	File Upload with Progress	8	Loss of Interactivity
9	Simulation & Scientific Modeling	9	Parallax Scrolling	9	Semantic Error
10	Infrastructure & System Mgmt.	10	Page Transitions	10	Nesting Error
11	DevTools & Engineering	11	Particle Effects	11	Missing Attributes
12	Logic & Workflow Visualization	12	Skeleton Loading		
13	Location Services & Transit	13	Shopping Cart		
14	Information & Personal Branding	14	User Authentication		
15	Lifestyle & Niche Utilities	15	Multi-step Wizard		
		16	Notification Center		

Our benchmark comprises a total of 1526 tasks, distributed as follows: 123 for Text-Guided Generation, 109 for Vision-Guided Generation, 94 for Video-Guided Generation, 300 for Text-Guided Editing, 300 for Vision-Guided Editing, 300 for Diagnostic Repair, and 300 for Visual-Diagnostic Repair. Each task is annotated with a difficulty level (Easy, Medium, or Hard) based on the complexity of the required functionality, the number of interactive components, and the sophistication of the visual design. A detailed breakdown of per-category counts is provided in Figure 2.

2.5 Task Type Descriptions

To comprehensively evaluate models across a wide spectrum of real-world web development challenges, WebCompass defines 15 generation application domains, 16 diverse editing task types, and 11 repair defect types. Table 2 provides an overview, and the following subsections detail each editing and repair task type.

2.5.1 Editing Task Types

The 16 editing task types span from low-level UI components to full business workflows, ensuring broad coverage of frontend engineering skills. They are organized into four categories:

Complex Components. This category includes *Data Table* (sortable, paginated, filterable table with row selection and inline editing), *Rich Text Editor* (WYSIWYG editor with formatting toolbar, link/image insertion, and form-synced output), *Drag & Drop Interface* (draggable items with drop-zone feedback, cross-container reordering, and state persistence), and *Tree View* (nested expand/collapse tree with cascading selection and search filtering).

Frontend-Backend Integration. This category covers *Real-time Dashboard* (live-updating metric cards with animated counters and sparkline charts), *Infinite Scroll* (scroll-triggered lazy loading with skeleton placeholders and end-of-content handling), *Async Form Validation* (debounced server-side validation with inline status indicators and submit gating), and *File Upload with Progress* (drag-and-drop upload with per-file progress bars, queue management, and cancel support).

Advanced Animations. This category encompasses *Parallax Scrolling* (multi-layer differential scroll speeds with viewport-triggered fade/scale effects), *Page Transitions* (coordinated enter/exit animations

such as fade, slide, and zoom between SPA content views), *Particle Effects* (canvas-based particle system with physics, cursor interaction, and connection lines), and *Skeleton Loading* (shimmer-animated placeholders matching content structure with smooth reveal).

Business Scenarios. This category includes *Shopping Cart* (full cart flow with quantity controls, real-time totals, and localStorage persistence), *User Authentication* (login, registration, and password-recovery forms with validation and auth state management), *Multi-step Wizard* (step indicator with per-step validation, cross-step data persistence, and review summary), and *Notification Center* (notification dropdown with unread badge, categorized alerts, and mark-as-read actions).

2.5.2 Repair Defect Types

The 11 repair defect types cover visual, semantic, and interactive failure modes commonly encountered in frontend development, organized into three dimensions:

Visual Layout. This dimension includes seven defect types: *Occlusion* (one element covers another due to incorrect z-index stacking), *Crowding* (spacing between elements is removed or collapsed, causing visual clutter), *Text Overlap* (text overflows its container and overlaps with adjacent content), *Alignment* (elements are offset from their expected grid or sibling alignment), *Color & Contrast* (text color is too close to the background, reducing readability), *Overflow* (content exceeds a fixed-size container without proper overflow handling), and *Sizing/Proportion* (elements are given extreme or distorted dimensions).

Semantic Correctness. This dimension includes *Semantic Error* (semantic HTML tags replaced with non-semantic equivalents, e.g., `<h1>` replaced by `<div>`) and *Nesting Error* (invalid HTML nesting, e.g., `<a>` inside `<a>`, or `<div>` inside `<p>`).

Interactive Usability. This dimension includes *Loss of Interactivity* (interactive elements disabled or blocked via pointer-events: none) and *Missing Attributes* (accessibility or functional attributes removed, e.g., alt, aria-label).

3 Evaluation Methodology

We adopt task-specific evaluation paradigms tailored to the output characteristics of each task family. For **Editing & Repair**, where models produce localized code patches, we use *LLM-as-a-Judge* (§3.1). For **Generation**, where correctness depends on end-to-end runtime behavior, we use *Agent-as-a-Judge* (§3.2). Both paradigms score along three axes—*executability*, *functional*, and *visual*—whose operationalization is task-dependent. For **Generation**: *Runnability* (build and launch success), *Spec Implementation* (functional behavior matches the design document), and *Design Quality* (visual polish). For **Editing**: *Instruction Targeting* (patch applies and targets the instruction’s required locations), *Feature Integrity* (original interactions preserved and new components functional), and *Style Conformance* (visual edit landed and unchanged regions consistent). For **Repair**: *Root-Cause Targeting* (patch applies and targets the defect’s root cause), *Interaction Integrity* (interactions preserved and interactive-class defects repaired), and *Reference Fidelity* (visual match to the ground-truth fixed screenshot). We select the judge model based on highest agreement with human annotations (§4.3.1).

3.1 LLM-as-a-Judge for Editing & Repair

For each instance, we apply the predicted patches to the source repository, discard blocks that fail to apply, and launch the modified project in a headless Chromium browser to capture screenshots (Figure 5). The judge receives the original task requirement, the source repository, the model-generated patch, build and runtime logs after patch application, and before/after screenshots captured in headless Chromium. For **Repair** tasks, it additionally receives the defect description, the ground-truth modifications, and the reference fixed screenshot. It scores checklist items independently along the three task-specific dimensions (0–10 each), produces evidence-grounded structured JSON output, and aggregates the resulting dimension-wise scores into the final task score.

3.2 Agent-as-a-Judge for Generation

Traditional evaluation approaches for web generation fall into two camps, each with a critical blind spot. Pure test-based methods (e.g., unit tests or DOM assertions) can programmatically verify functional correctness—whether a button triggers the right callback or a form validates inputs—but cannot assess

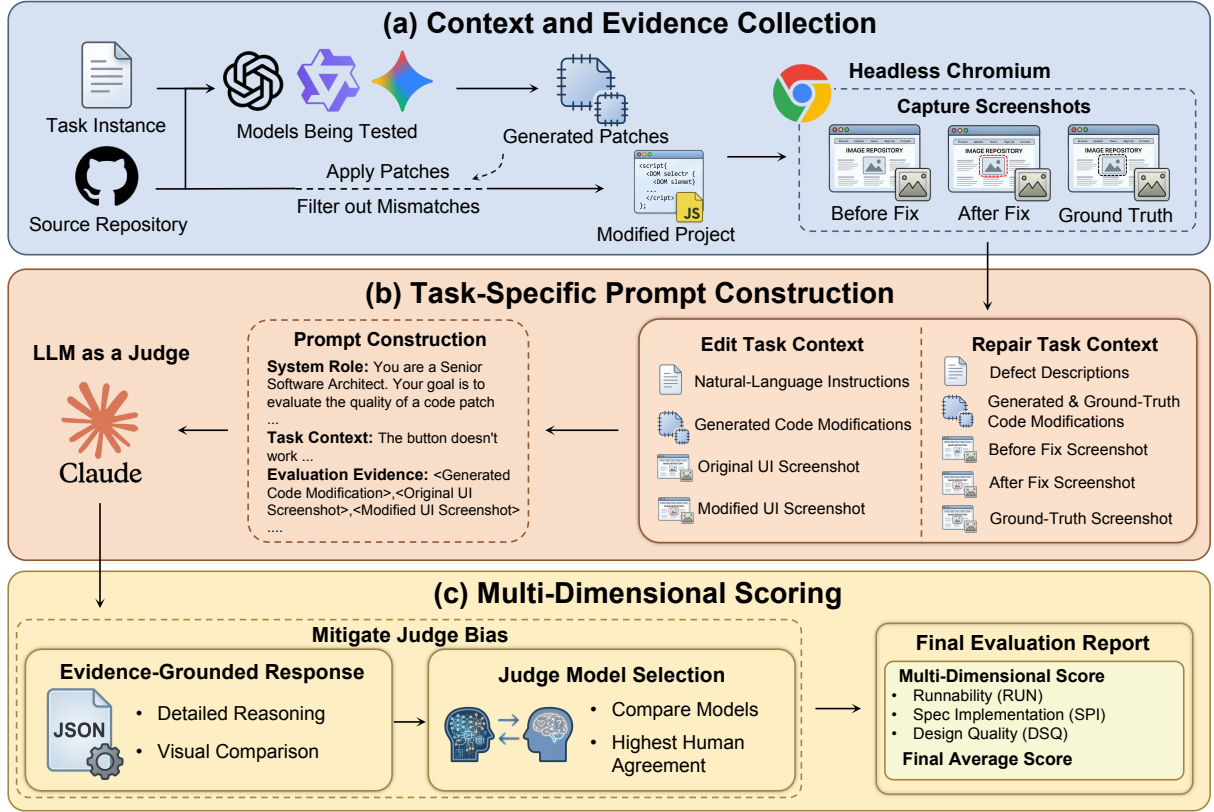


Figure 5: Illustration of the LLM-as-a-Judge evaluation pipeline.

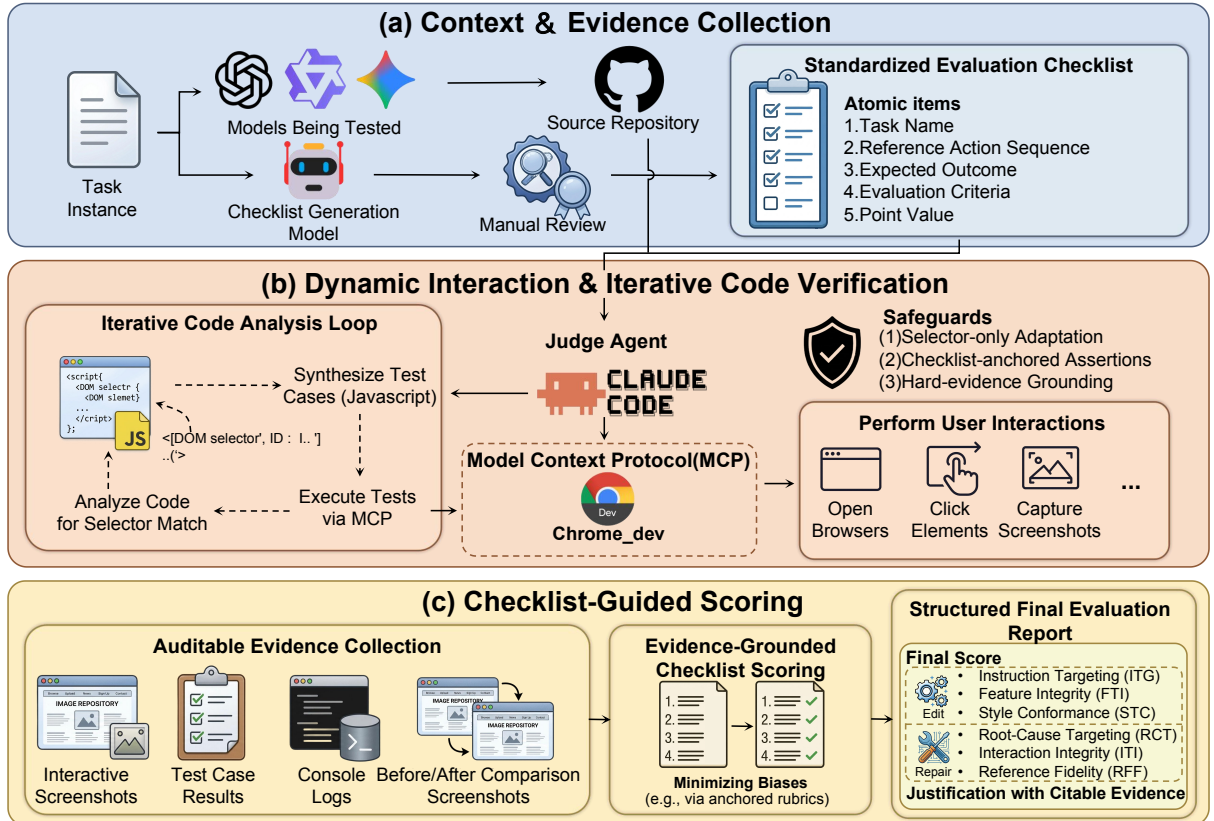


Figure 6: Agent-as-a-Judge evaluation pipeline. The MCP bridge enables bidirectional communication: the agent sends interaction commands to the browser and receives DOM snapshots, console logs, and screenshots as evidence.

visual fidelity, layout harmony, or aesthetic quality. Conversely, screenshot-based comparison methods can capture visual appearance but struggle to verify multi-step interactive behaviors, state transitions, and dynamic content that only manifest through real user interaction. Human acceptance testing naturally combines both capabilities: a QA engineer can inspect the UI visually *and* write ad-hoc test scripts to probe edge cases, switching fluidly between the two modes. To approximate this dual capability in an automated setting, we adopt **Claude Code** as the evaluation orchestrator paired with the **Model Context Protocol (MCP)** for browser control. This architecture is deliberately chosen because it endows the judge agent with two complementary verification channels: (1) as a code agent, it can dynamically *synthesize and execute JavaScript test cases* that programmatically inspect DOM states, CSS properties, and functional logic with deterministic precision; and (2) through the MCP bridge to a real browser, it can *simulate authentic user interactions*—clicking, scrolling, typing, navigating—while capturing screenshots and console logs as auditable evidence. Neither channel alone suffices: scripted tests miss visual quality, and browser interaction alone cannot efficiently verify complex state invariants. Their combination enables a unified evaluation loop that closely mirrors how a human tester would accept or reject a web application.

Figure 6 illustrates our *Agent-as-a-Judge* pipeline. A Code Agent, augmented with the Model Context Protocol (MCP) for real-browser control, evaluates each generated website in four stages:

(1) Checklist generation: an LLM produces a structured evaluation checklist defining tasks, interaction sequences, expected outcomes, and score values; this checklist remains fixed throughout to prevent circular reasoning.

(2) Browser interaction: the agent launches the website in headless Chromium, executes checklist interactions (clicking, typing, scrolling, navigation), and records DOM snapshots, console logs, and screenshots as auditable evidence.

(3) Adaptive code verification: the agent synthesizes executable JavaScript test cases for each checklist item, programmatically verifying DOM states, CSS properties, and functional behaviors. Crucially, when implementation details differ from expectations, the agent adapts only DOM locators (e.g., element selectors and IDs) while keeping all behavioral assertions unchanged—ensuring that evaluation criteria remain anchored to the original specification rather than drifting toward the model’s output. Failed tests trigger an iterative debugging loop in which the agent inspects the actual code, diagnoses the mismatch, and re-attempts verification before assigning a score.

(4) Evidence-grounded scoring: the agent scores each item along Runnability, Spec Implementation, and Design Quality with structured justifications; scores lacking auditable evidence (screenshots, test results, or console logs) are discarded.

Three safeguards prevent evaluation bias: checklist immutability (no new criteria after Stage 1), selector-only adaptation in Stage 3, and mandatory hard-evidence grounding for every score.

All experiments are conducted on a Linux server with per-task execution timeouts to prevent infinite loops or hanging processes. For generation evaluation, we use Claude Code (v2.0.67) as the evaluation orchestrator together with the Chrome DevTools MCP Server (v0.19.0), which provides headless Chromium rendering, DOM inspection, and browser automation capabilities.

3.3 Scoring and Failure Handling

Scoring formula. For each task instance, let $\{s_1, s_2, \dots, s_n\}$ denote the individual checklist item scores and $\{m_1, m_2, \dots, m_n\}$ their corresponding maximum scores. Each item’s normalized score is $r_i = s_i / m_i$. To prevent a single zero-scored item from collapsing the entire task score, we apply a smoothing constant $\epsilon = 1$ to any item where $s_i = 0$, replacing its normalized score with ϵ / m_i . The task-level score is then computed as the harmonic mean of the normalized item scores:

$$s_{\text{task}} = \frac{n}{\sum_{i=1}^n \frac{1}{r_i}} \quad (1)$$

We choose the harmonic mean over the arithmetic mean because it penalizes imbalanced performance: a web artifact that excels on some criteria while completely failing on others should not receive a high overall score. This score is computed separately for each of the three per-task evaluation dimensions.

Handling cascading failures. Web generation tasks frequently produce outputs that fail at different stages of the build–render–interact pipeline, and a naïve application of the scoring formula could yield misleading results. We define explicit fallback strategies for three failure scenarios: **1. Complete build failure** (the project does not compile or launch): the functional and visual dimensions are set to 0; only the executability dimension contributes a meaningful score. **2. Partial rendering failure** (the project launches but some pages or components fail to render): the executability dimension is penalized proportionally;

the visual dimension is evaluated on the rendered portion (or set to 0 if nothing is visible); the functional dimension is evaluated only on reachable components. **3. Runtime crash** (the project renders initially but crashes during interaction): the executability and visual dimensions are scored on the initial render; the functional dimension is scored only on the testable subset, with untestable items receiving 0. These fallback rules ensure that cascading failures degrade scores gracefully rather than producing undefined or inflated results, faithfully reflecting the progressive nature of web application quality.

4 Experiments

This section describes our experimental setup and provides a comprehensive overview of results. We organize experiments by **task type** (generation / editing / repair) and **input modality** (text / image / video). Beyond the main results (§4.2), we report several focused analyses: judge model selection (§4.3.1), framework-based subset evaluation (§4.3.2), difficulty-level analysis (§4.3.4), and impact of thinking mode (§4.3.8).

4.1 Evaluated LLMs and Frameworks

We report main benchmark results for ten models from both closed-source and open-source families. All selected models natively support text, image, and video inputs, allowing us to use the same model set across all modalities. Full model details, including auxiliary comparison variants used in later analyses, are provided in Appendix A.4.

We employ **Claude Code** (v2.0.67) as the evaluation orchestrator and the **Chrome DevTools MCP Server** (v0.19.0) for browser rendering, DOM inspection, and automated interaction verification in a headless Chromium environment. This setup enables an agent-based evaluation pipeline that programmatically assesses the functional correctness and UI consistency of generated web applications.

4.2 Main Results

Table 3 presents the overall and per-task-type scores for all evaluated models.

Table 3: Comparison of models across different task types. **Green bold** indicates the best score in each column; **blue underline** indicates the second best. Each task has three evaluation dimensions: Generation uses Runnability (RUN), Spec Implementation (SPI), and Design Quality (DSQ); Editing uses Instruction Targeting (ITG), Feature Integrity (FTI), and Style Conformance (STC); Repair uses Root-Cause Targeting (RCT), Interaction Integrity (ITI), and Reference Fidelity (RFF). Overall is the arithmetic mean of all nine dimension scores.

Models	Generation			Editing			Repair			Overall
	RUN.	SPI.	DSQ.	ITG.	FTI.	STC.	RCT.	ITI.	RFE.	
Closed-Source Large Language Models										
Claude-Opus-4.5	77.18	68.95	62.26	71.86	65.82	60.83	48.45	85.54	65.71	67.40
Gemini-3-Pro-Preview	74.05	55.76	64.07	69.52	65.14	58.16	54.16	87.30	72.00	66.68
Gemini-3-Flash-Preview	74.87	54.32	62.42	65.95	62.35	57.21	53.18	86.84	71.65	65.42
GPT-5.2	75.38	60.22	55.92	66.97	62.70	56.63	41.24	79.33	58.70	61.90
Claude-Sonnet-4.5	65.30	50.37	56.78	60.06	53.71	45.51	40.44	80.63	61.31	57.12
Qwen3-VL Series Open-Source Large Language Models										
235B-A22B-Instruct	61.26	42.14	47.06	27.74	25.48	23.53	27.30	68.87	46.88	41.14
235B-A22B-Thinking	63.86	35.02	45.21	22.15	21.67	19.06	27.02	68.74	46.28	38.78
32B-Instruct	50.39	25.62	34.56	26.96	26.62	22.78	24.67	61.93	43.27	35.20
30B-A3B-Thinking	47.37	20.87	37.47	19.82	21.21	18.20	18.08	51.85	31.31	29.58
30B-A3B-Instruct	41.79	20.80	29.28	20.57	20.97	17.93	19.32	50.71	31.35	28.08

Several key patterns emerge.

Model ranking and the closed-open gap. Claude-Opus-4.5 and Gemini-3-Pro-Preview achieve the highest Overall scores (67.40 and 66.68, respectively) with complementary strengths: Claude leads

Generation RUN (77.18) and Editing ITG (71.86), while Gemini leads Repair RCT (54.16) and Repair RFF (72.00). The closed–open gap is substantial: the best open-source model (Qwen3-VL-235B-A22B-Instruct) reaches an Overall of 41.14, trailing the top closed-source model by over 26 points. Smaller open models (30B variants) fall further, reaching less than half the top closed-source scores.

Task-type patterns. For closed-source models, Generation and Editing consistently follow the ordering executability > functional > visual (e.g., Claude-Opus-4.5: RUN 77.18 > SPI 68.95 > DSQ 62.26 on Generation). Repair shows a different pattern: ITI $\gg$ RFF > RCT (e.g., Gemini-3-Pro-Preview: 87.30 > 72.00 > 54.16). This ordering is explained by the task structure: Interaction Integrity trends high because 9 of 11 defect types are visual or semantic—the interactive layer is rarely affected, so preservation is nearly automatic; the 2 interactive-class defects (Loss of Interactivity, Missing Attributes) are localized enough that models can usually repair them. Reference Fidelity is mid-range because matching the gold fixed screenshot is nontrivial. Root-Cause Targeting is lowest because correctly locating the defect’s root cause without introducing new errors remains the hardest part of repair. Note that the functional and visual axes measure different capabilities across tasks: Editing’s Feature Integrity tests both preservation and new-component functionality, whereas Repair’s Interaction Integrity is primarily regression safety; similarly, Editing’s Style Conformance evaluates edit outcome fidelity, while Repair’s Reference Fidelity measures closeness to a gold reference. Editing is especially challenging for open-source models, where scores fall to 18–28 across dimensions, revealing a major gap in context-aware code modification.

Visual quality as the persistent bottleneck. Across all ten models, the visual dimension is the lowest-scoring axis in Generation and Editing (Design Quality and Style Conformance, respectively). Even Gemini-3-Pro-Preview, the strongest model on this axis, reaches only 64.07 on Generation DSQ. The gap is wider for weaker models and consistent across task types. Notably, Gemini-3-Pro-Preview and Gemini-3-Flash-Preview outperform GPT-5.2 on the visual axis despite comparable executability scores, indicating that visual fidelity and functional correctness do not scale in lockstep.

4.3 Further Analysis

4.3.1 Judge Model Selection

To validate automated evaluation reliability, we compare three Claude-family judge models (Opus-4.5, Sonnet-4.5, Haiku-4.5) against human judgments on a 200-sample subset. As shown in Table 4, Claude-Opus-4.5 achieves the highest human agreement (Pearson r of 0.93–0.96 across tasks), and is adopted as the default judge. Notably, all judges show higher agreement on edit/repair tasks than on generation, consistent with the more constrained solution space of patch-based tasks. As shown in Figure 7, a comparison of full model rankings between the agent-based evaluator and human annotators further confirms strong alignment, with most rank differences being zero or at most one, validating the automatic evaluation protocol as a reliable proxy for human judgment.

Table 4: Judge model comparison. We report human agreement (Pearson r) and estimated cost per sample. **Green bold:** best; **blue underline:** second best.

Judge Model	Generation	Editing	Repair	Cost Analysis
	r	r	r	Cost
Claude-Opus-4.5	0.93	0.94	0.96	\$4.66
Claude-Sonnet-4.5	<u>0.88</u>	<u>0.90</u>	<u>0.89</u>	\$2.34
Claude-Haiku-4.5	0.76	0.79	0.81	\$1.02

This comparison also reveals a clear cost–quality trade-off. The cost column in Table 4 reports the average API token expenditure (in USD) for evaluating a single task instance. Claude-Sonnet-4.5 is cheaper but consistently trails Opus in agreement, while Haiku shows a substantial drop in alignment despite the lowest cost. We therefore choose Opus-4.5 as the default judge because judge reliability is foundational to benchmark validity, and the additional evaluation cost is justified by the stronger agreement with human assessment.

4.3.2 Subset Evaluation on Different Front-End Frameworks

To assess how framework choice affects model performance, we evaluate four representative models (GPT-5.2, Gemini-3-Pro-Preview, Claude-Opus-4.5, Qwen3-VL-235B-A22B-Instruct) on a 180-task subset (60 per task category), each completed in **React**, **Vue**, and **Vanilla** (plain HTML/CSS/JS). Figure 8 presents the overall scores; per-dimension breakdowns are in Appendix A.3.

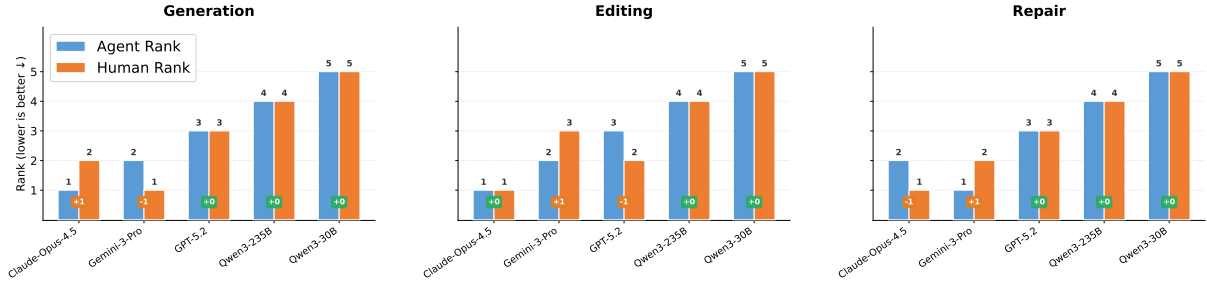


Figure 7: Comparison of model rankings between agent-based automatic evaluation and human evaluation across three tasks. In most cases, the rank difference is zero or at most one, indicating strong agreement between the automatic evaluator and human annotators.

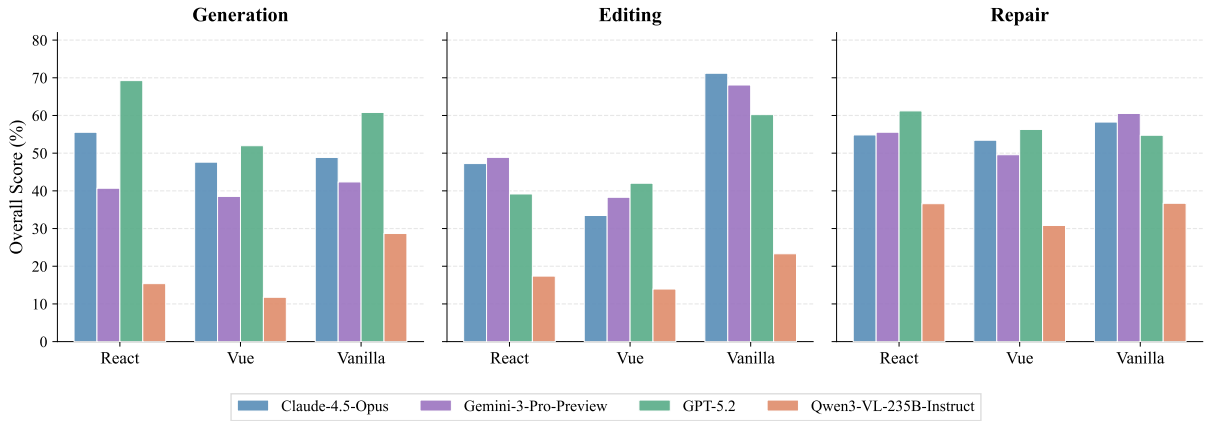


Figure 8: Overall scores across front-end frameworks for four representative models on Generation, Editing, and Repair tasks. Scores are computed as the harmonic mean of the three per-task evaluation dimensions.

Three key findings emerge. **(1) Vanilla dominates Generation and Editing, but not Repair.** Across all four models, framework-free code consistently yields the highest scores in Generation and Editing. In Repair, however, the Vanilla advantage diminishes: for instance, GPT-5.2 achieves its best Repair score with React. We attribute this to a structural difference between task types: Generation and Editing require producing substantial new code, where Vanilla’s absence of build toolchains, framework-specific syntax (e.g., JSX, template directives), and component lifecycle conventions reduces the surface area for errors. Repair, by contrast, demands precise localization and modification of existing defects, where React’s explicit component boundaries and unidirectional data flow may help models isolate faulty code regions more effectively than unstructured Vanilla codebases. **(2) Vue consistently underperforms.** Vue yields the lowest scores in the majority of model–task combinations. A plausible contributing factor lies in Vue’s single-file component (SFC) format, which interleaves three heterogeneous syntax modes—HTML-like templates with custom directives (`v-if`, `v-for`, `@click`), JavaScript/TypeScript logic, and scoped CSS—within a single file. This demands simultaneous coordination across markup, logic, and styling, increasing the likelihood of cross-block inconsistencies. By comparison, React’s JSX keeps rendering logic within standard JavaScript, and Vanilla avoids framework abstractions entirely. **(3) Open-source models share the same framework sensitivity pattern** (peaking on Vanilla for Generation/Editing) but at a uniformly lower performance ceiling, suggesting that the observed framework preferences are primarily driven by inherent task–framework interactions rather than model-specific factors.

4.3.3 Task-Type Breakdown

To reveal where strong models succeed and fail, we further decompose Edit and Repair into fine-grained subtask categories for the three best closed-source models. Figures 9 and 10 report the harmonic-mean score for each subtask type.

Editing: animation-heavy operations are the hardest. A clear difficulty hierarchy emerges across editing operation types (Figure 9): Business Scenario tasks such as Shopping Cart and Multi-step Wizard are consistently the easiest, followed by Real-time & Async tasks, then Interactive Components, with

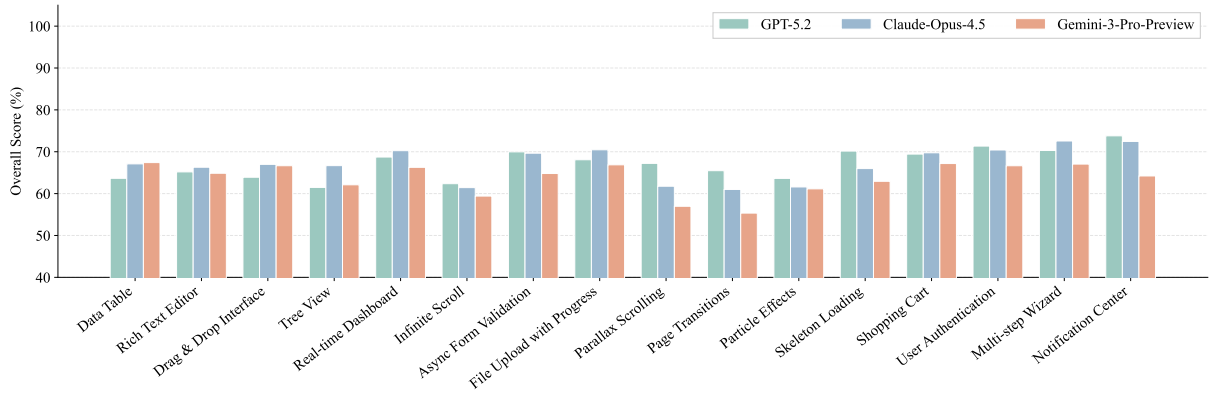


Figure 9: Overall score breakdown for editing tasks across 16 operation types. Scores are computed as the harmonic mean of Instruction Targeting, Feature Integrity, and Style Conformance per subtask, averaged over all instances containing that operation type.

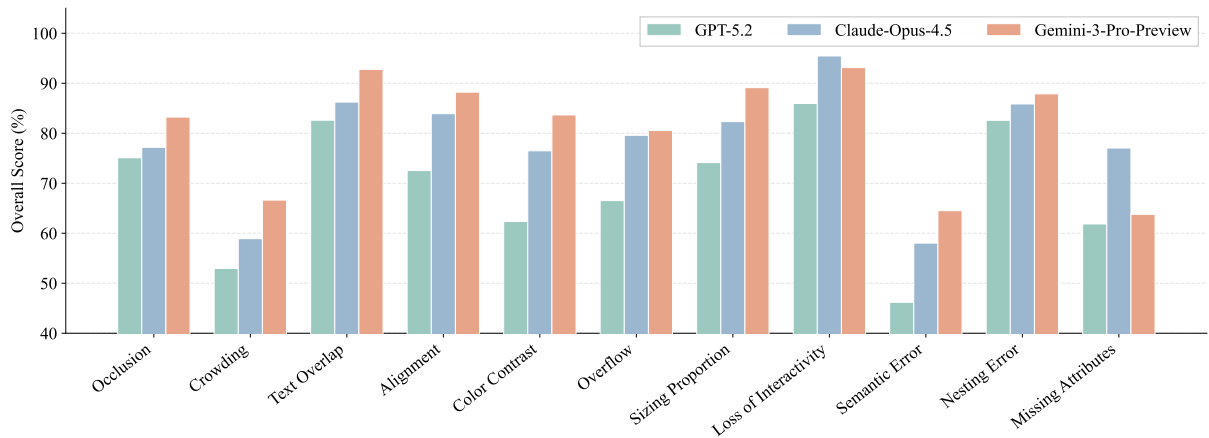


Figure 10: Overall score breakdown for repair tasks across 11 defect categories. Scores are computed identically to Figure 9.

Advanced Animation tasks such as Parallax Scrolling, Page Transitions, and Particle Effects forming the hardest category. This ordering is stable across all three models, suggesting that editing difficulty scales with the degree of visual dynamism and cross-component coordination required.

Repair: semantic defects remain the main bottleneck. A similar difficulty gradient appears in repair (Figure 10). Structural and interactive defects such as Loss of Interactivity, Nesting Error, and Text Overlap are reliably fixed, as they often manifest in localized DOM structures or event handlers. Semantic-level defects, however, prove substantially harder: Semantic Error elicits the lowest scores across all models, followed by Crowding and Missing Attributes. These categories require reasoning about design intent and implicit visual constraints that go beyond pattern-matching on code structure.

Consistency matters more than isolated wins. An instructive discrepancy emerges for editing: GPT-5.2 outperforms Gemini-3-Pro-Preview on 13 of 16 subtask types when averaged per category, yet trails on instance-level scores in Table 3. This reversal stems from our harmonic-mean aggregation—GPT-5.2 exhibits higher cross-subtask variance, and the harmonic mean penalizes low outliers steeply. This highlights that multi-requirement evaluation rewards consistency, not just peak subtask performance. No such reversal occurs for repair, where Gemini-3-Pro-Preview leads on all 11 defect categories, reflecting genuinely superior repair capability.

4.3.4 Difficulty-Level Analysis

Each WebCompass instance is annotated as *Easy*, *Medium*, or *Hard* according to required functionality, number of interactive components, and visual sophistication. This stratification lets us examine how model quality degrades with task complexity.

To further investigate how model capabilities scale with task complexity, we break down the evaluation results across three difficulty levels (*Easy*, *Medium*, *Hard*) for each task category. Figure 11 presents an overview across all three task families. Across all task families and evaluation dimensions, model scores consistently decrease as difficulty increases (Figures 12, 13, and 14). This degradation is particularly striking in generation on the *Spec Implementation* dimension (Figure 12), where Hard tasks require implementing more complex user flows, multi-step state transitions, and richer dynamic behavior. For example, Gemini-3-Pro-Preview drops from 89.83 on Easy generation tasks to 37.64 on Hard ones, suggesting that faithfully implementing the full functional spec becomes disproportionately challenging as task complexity grows.

Cross-task observations. As shown in Figure 11, Qwen3-VL-235B-A22B-Instruct consistently ranks last across difficulty-task combinations, with a particularly large gap on Hard tasks. More broadly, the performance gap between top-tier proprietary models and weaker open models widens as difficulty increases, suggesting that stronger reasoning capabilities become increasingly critical for complex front-end tasks.

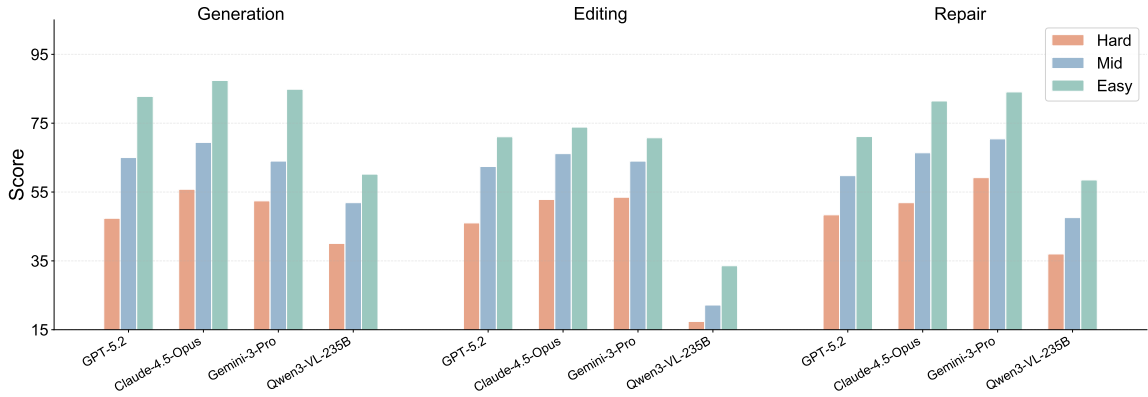


Figure 11: Performance comparison across Generation, Editing, and Repair tasks by difficulty level.

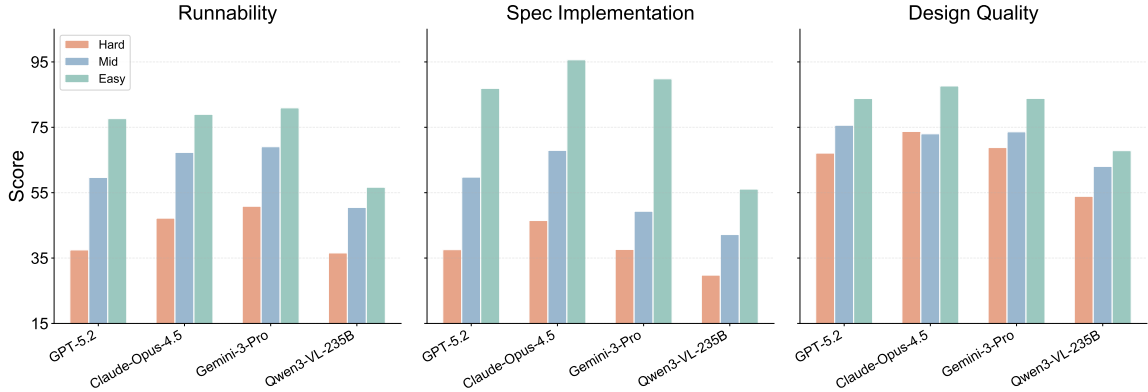


Figure 12: Generation task: per-dimension scores (Runnability, Spec Implementation, and Design Quality) across three difficulty levels. Each model has three bars representing Hard (red), Medium (blue), and Easy (green).

4.3.5 Patch Complexity Analysis

Beyond output quality, we analyze the structural complexity of model-generated patches using two complementary metrics: **Changed Lines** (added plus deleted lines) and **Patch Count** (number of contiguous diff hunks). For repair, we additionally compare against the human-authored ground-truth patches.

Figure 15 reveals two patterns. First, editing patches are far larger than repair patches, matching the task structure: editing often introduces new components or rewrites existing interaction flows, whereas repair usually targets localized defects. Edit tasks yield median patch sizes of 646–1,976 changed lines across models, while repair patches are much smaller, with medians of 16–19 lines. Second, stronger models do not simply generate larger patches. Claude-Opus produces the largest edit patches, roughly three times larger than Gemini models, despite only modest quality differences, while repair patches stay close to

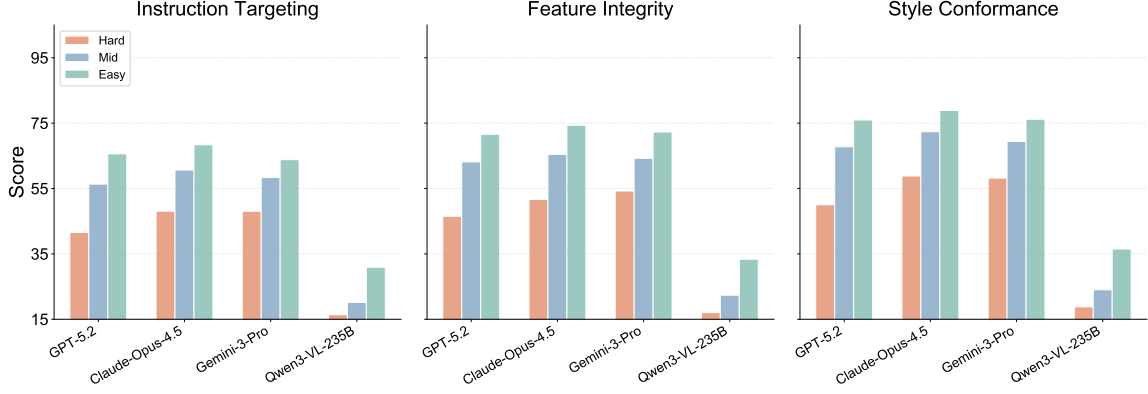


Figure 13: Edit task: per-dimension scores (Instruction Targeting, Feature Integrity, and Style Conformance) across three difficulty levels.

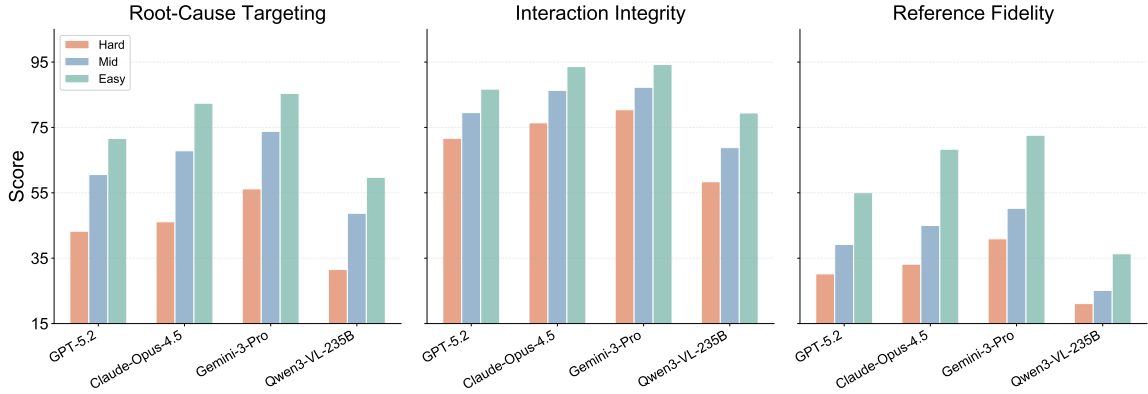


Figure 14: Repair task: per-dimension scores (Root-Cause Targeting, Interaction Integrity, and Reference Fidelity) across three difficulty levels.

the human-authored baseline but exhibit heavier right tails, indicating occasional over-editing. Together these results suggest that successful web coding depends less on patch size itself than on targeting the right code regions with coherent, well-localized updates.

4.3.6 Stability Analysis: Worst-of-N Evaluation

Pass@1 reflects average-case capability but may mask output inconsistency. We adopt the Worst-of- n (W@ n) protocol, sampling $n = 4$ independent generations per task and reporting the minimum score to capture the realistic lower bound of model performance.

As shown in Figure 16, both models degrade monotonically from Pass@1 to W@4, but at different rates. **Gemini-3-Pro-Preview** retains $\sim 80\%$ of its Pass@1 performance at W@4 (66.96 $\rightarrow$ 53.56), with all dimensions remaining above 49%. **Qwen3-VL-235B-A22B-Instruct** degrades more sharply, retaining only $\sim 69.5\%$ (39.95 $\rightarrow$ 27.78), with W@4 scores in the Edit category falling below 16% on Instruction Targeting—indicating near-complete failure in worst-case scenarios.

These results reveal that Gemini’s advantage extends beyond higher average scores to *greater output consistency*. Since users typically rely on a single generation rather than selecting from multiple samples, output stability remains a critical open challenge for frontier models in front-end code generation.

4.3.7 Text-Only vs. Vision-Language Models

To investigate whether visual grounding helps or hurts front-end code generation when the task itself is text-based, we compare QWEN3-32B and QWEN3-VL-32B-INSTRUCT on three representative text-only task types: Text-Guided Generation, Text-Guided Editing, and Diagnostic Repair.

Complementary strengths. The comparison reveals a non-trivial trade-off. QWEN3-VL-32B-INSTRUCT consistently outperforms QWEN3-32B on the visual axis across all three task types, suggesting that the vision-language model carries a stronger internal rendering prior that benefits layout and styling

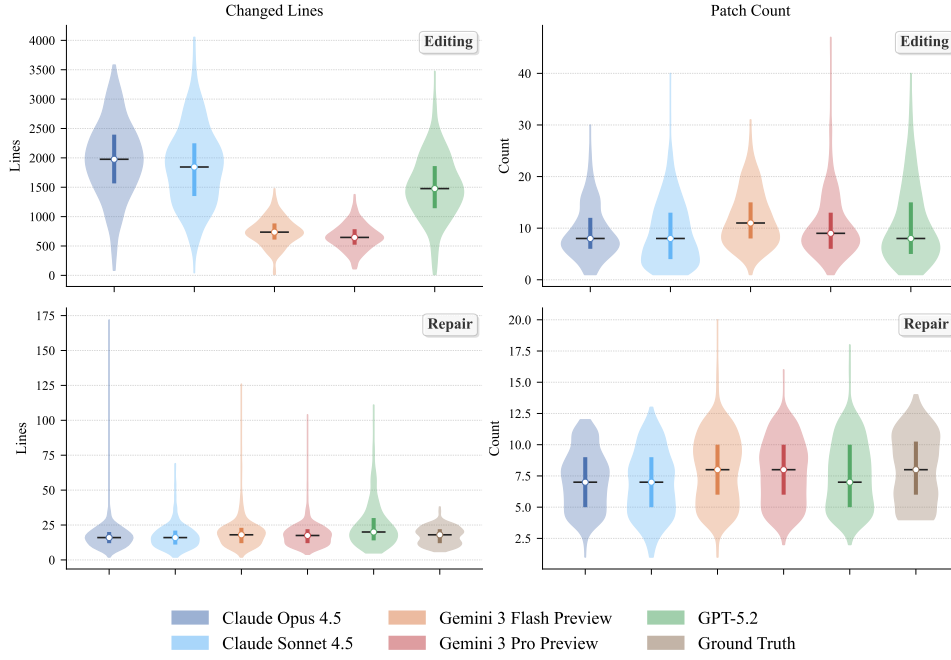


Figure 15: Distribution of patch complexity across models. Top row: Edit tasks; bottom row: Repair tasks (with Ground Truth baseline). Each violin shows the full distribution; the thick bar marks the interquartile range (Q1–Q3) and the white dot marks the median.

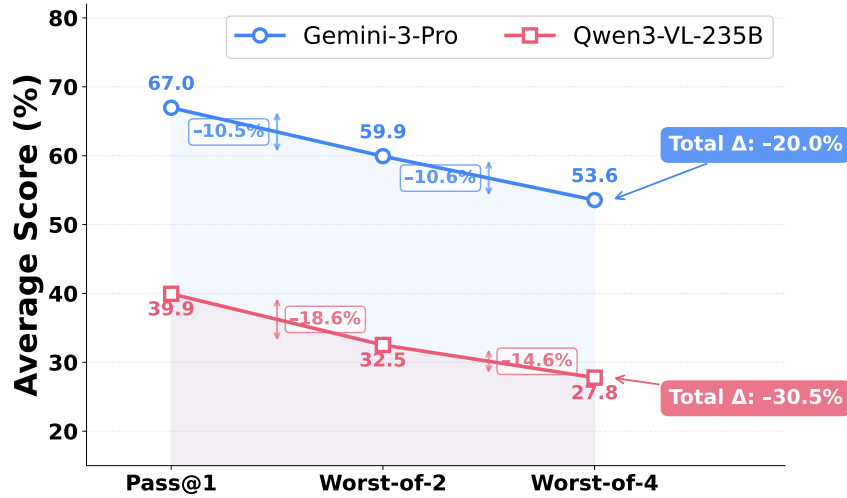


Figure 16: Consistency & Stability: Score Degradation under Worst-of-N

fidelity even on text-only tasks. Conversely, the text-only model retains an advantage on Generation Runnability, indicating more robust code synthesis in scenarios where success depends on clean functional implementation rather than visual grounding.

Overall, the two architectures exhibit complementary strengths: vision-language grounding benefits visual fidelity, while the text-only model can still retain an edge in producing functionally reliable interactive code. This result suggests that stronger multimodal perception does not automatically translate into uniformly better web coding performance, especially when the task is primarily constrained by code reasoning rather than visual reconstruction.

4.3.8 Impact of Thinking Mode on Performance

Recent work on reasoning-enhanced LLMs has introduced “thinking” or “chain-of-thought” modes that encourage models to reason step-by-step before producing a final answer (Guo et al., 2025). To investigate whether this paradigm benefits web development tasks, we compare the Instruct and Thinking variants

Table 5: Comparison of QWEN3-32B and QWEN3-VL-32B-INSTRUCT on the text-only subset across three web-development task types. **Green bold** indicates the best score in each column. Dimension abbreviations follow Table 3; Overall is the arithmetic mean of all nine dimension scores.

Models	Generation			Edit			Repair			Overall
	RUN.	SPI.	DSQ.	ITG.	FTI.	STC.	RCT.	ITI.	RFE.	
Qwen3-32B	56.28	6.50	49.10	21.52	21.17	17.92	21.42	56.73	33.72	31.60
Qwen3-VL-32B-Instruct	44.48	14.53	56.92	28.16	27.86	24.17	26.01	64.14	42.15	36.49

of two Qwen3-VL models that appear in our evaluation: Qwen3-VL-235B-A22B and Qwen3-VL-30B-A3B.

As shown in Table 3, the impact of thinking mode varies across task types and evaluation dimensions. On **Generation** tasks, both Thinking variants achieve higher Runnability scores than their Instruct counterparts (63.86 vs. 61.26 for 235B; 47.37 vs. 41.79 for 30B), indicating that chain-of-thought reasoning helps with code structural correctness. However, the 235B Thinking model suffers a notable Spec Implementation drop (35.02 vs. 42.14), while the 30B model shows negligible change (20.87 vs. 20.80). On **Edit** and **Repair** tasks, differences between Thinking and Instruct variants are relatively minor for both scales.

The limited impact on Edit and Repair tasks likely reflects that these tasks exceed the current capability boundary of Qwen3-VL models. As shown in Table 3, both models score substantially lower on Edit than on Generation (e.g., 27.74 vs. 61.26 on the executability dimension for the 235B Instruct), suggesting that accurately comprehending existing code, locating modification points, and producing precise changes poses a fundamental challenge. When the task difficulty surpasses the model’s base competence, thinking mode cannot compensate for the lacking skills—there is insufficient domain knowledge for the reasoning chain to meaningfully build upon.

For Generation tasks, where models perform relatively better, the Spec Implementation degradation of the 235B Thinking variant is notable. Our error analysis reveals that this model produces significantly more *Feature Missing* errors—cases where required interactive features are absent from the output. We attribute this to **attention dilution** caused by lengthy reasoning chains. Web development prompts often specify multiple requirements simultaneously—layout, styling, interactive behaviors, and responsiveness. The 235B model, with its greater capacity, generates substantially longer thinking chains than its 30B counterpart, pushing the original feature specifications far from the code generation point in the context window. This makes it easier for the model to overlook specific requirements, producing structurally sound but incomplete implementations. The 30B model’s shorter reasoning chains preserve proximity to the original prompt, explaining its stable Spec Implementation scores.

4.3.9 Generation Error Patterns

To understand not only how well but also *how* LLMs fail, we design a structured error analysis framework that classifies every point deduction into a two-level taxonomy spanning four domains (Code Execution, Functional, Visual/Style, and Non-Functional) with fifteen fine-grained error types, and further attributes each error to a root cause. Full taxonomy definitions, the classification prompt, and the decision flowchart are provided in Appendix A.6.1; an extended cross-task breakdown is presented in Section 4.3.10.

Across models, three failure modes dominate. **Feature Missing** is the most common generation error, especially on difficult prompts that combine layout, interaction, and styling constraints. **Visual inconsistency** remains pervasive even when code executes correctly, confirming the gap between functional correctness and aesthetic fidelity observed in Table 3. Finally, in repair settings, models often fix the visible symptom while missing the underlying semantic cause, which is consistent with the weak performance on semantic defect categories in the subtask analysis above.

More concretely, Feature Missing and Resource Fail together account for roughly 40%–55% of all generation errors across most models. Lower-ranked models accumulate many fundamental failures such as missing functionality and console errors, whereas stronger models reduce these basic failures and leave a larger share of finer-grained layout and styling issues. Distinct modality-specific patterns also emerge: text-conditioned generation is dominated by Feature Missing errors, indicating difficulty translating natural-language specifications into executable interaction logic; image-conditioned generation shifts toward layout, color, and visual-fidelity errors, exposing weakness in pixel-level reproduction; and video-conditioned generation exhibits a more balanced mix of functional and visual errors, reflecting the compound challenge of understanding temporal interaction sequences while faithfully reproducing static

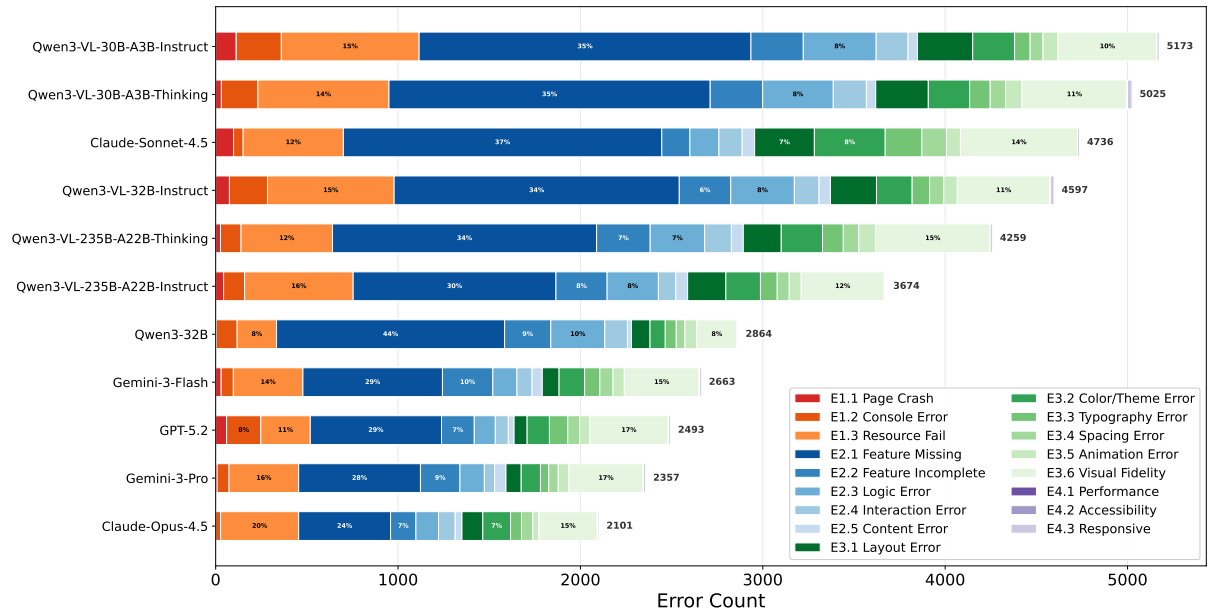


Figure 17: Overall error distribution across all evaluated models on web generation tasks. Feature Missing and Resource Fail dominate across models, while stronger models exhibit a larger fraction of finer-grained visual and styling errors after reducing fundamental execution failures.

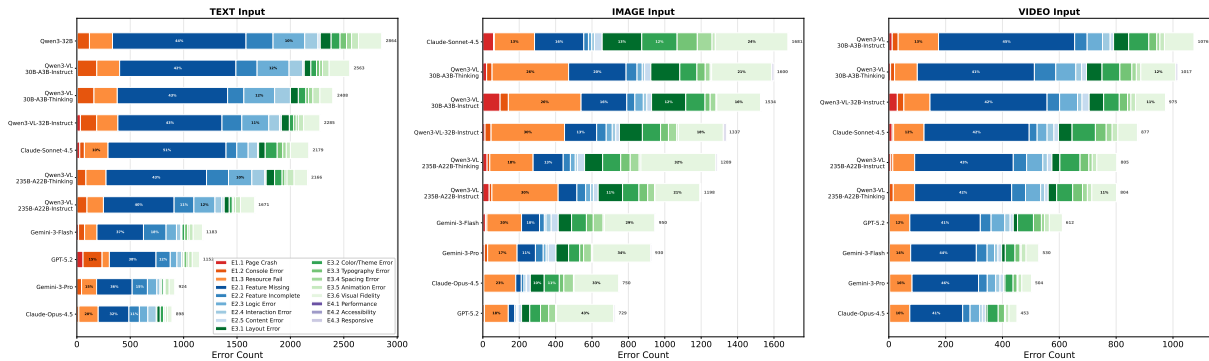


Figure 18: Error distribution by input modality. Text-conditioned generation is dominated by functional omissions, image-conditioned generation shifts toward visual fidelity and layout errors, and video-conditioned generation exhibits a balanced mix of functional and visual failures.

appearance. In other words, text primarily stresses requirement comprehension, images stress visual reconstruction, and videos simultaneously stress temporal reasoning and appearance matching, making them the most compositionally demanding input modality.

4.3.10 Editing and Repair Error Patterns

The overview above establishes the dominant generation-side failure modes and modality-specific patterns. We next extend the analysis with task-specific quantitative error distributions for Edit and Repair, revealing where patch-based models fail beyond the generation setting.

Quantitative Error Distribution in Edit and Repair Tasks. To complement the qualitative observations above, we conduct a comprehensive quantitative breakdown of error frequencies across Edit and Repair tasks. By analyzing the automated checklist deduction logs, we categorize the failures into specific sub-types. Figures 19 and 20 illustrate the total error counts and their proportional distributions across models for Edit and Repair tasks, respectively.

Several striking patterns emerge from this quantitative lens:

- **Edit Tasks are Bottlenecked by Feature Completeness and Logic (E2).** As shown in Figure 19, the vast majority of errors in editing tasks stem from the *Feature Missing* (E2.1) and *Feature Incomplete*

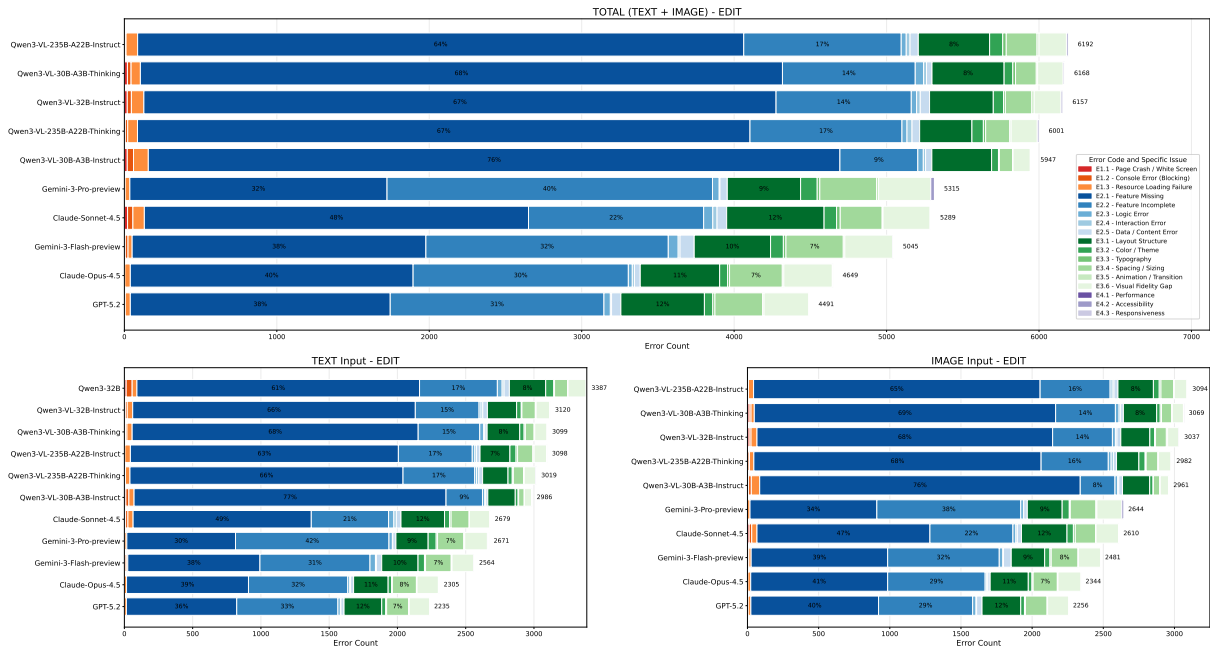


Figure 19: Quantitative distribution of error types for **Edit** tasks. The errors are categorized into Blocking/Crash (Orange/Red), Logic & Features (Blue), Visual & Layout (Green), and Accessibility/Performance (Purple). The total error count for each model is displayed on the right.

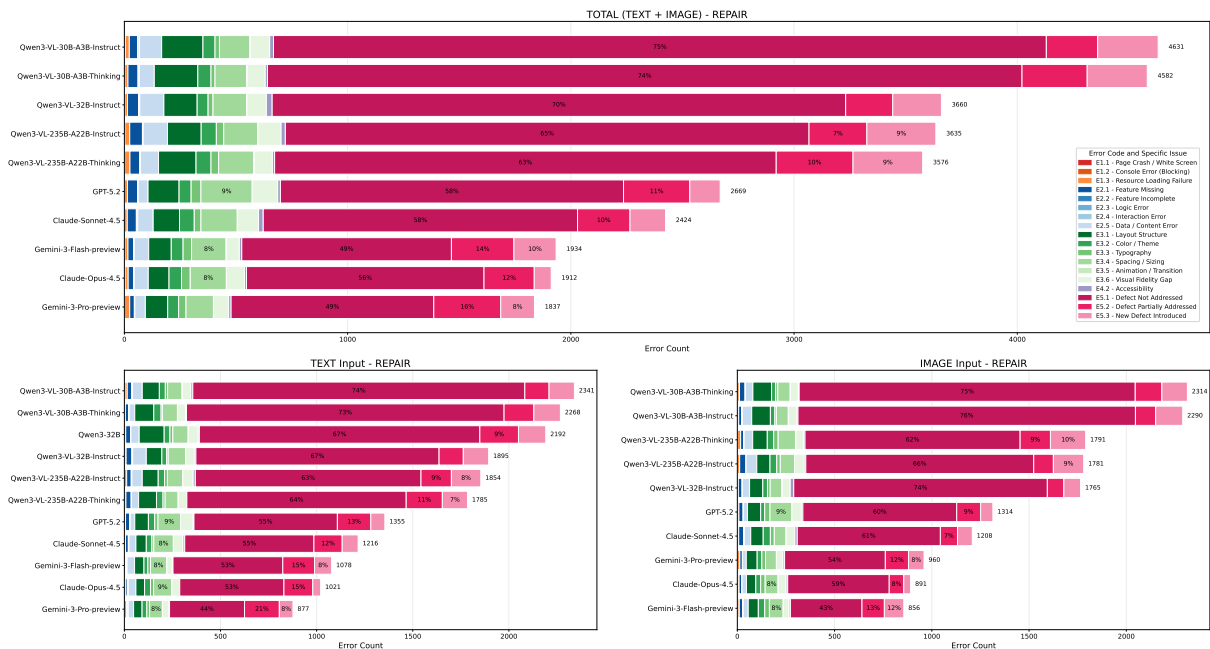


Figure 20: Quantitative distribution of error types for **Repair** tasks. In addition to standard web errors, Repair tasks introduce defect-resolution specific errors (Pink/Magenta): Defect Not Addressed, Partially Addressed, and New Defect Introduced.

(E2.2) categories (represented in dark and medium blue). For open-source models like Qwen3-VL-30B-A3B-Instruct, E2.1 alone accounts for up to 76% of all checklist failures. Even for top-tier closed-source models, these logical and feature-level omissions dominate (e.g., Claude-Opus-4.5: 40% E2.1 and 30% E2.2). This aligns with our qualitative finding that models often suffer from “partial implementation,” losing track of complex or multi-step editing instructions.

- **Visual Fidelity (E3) is the Secondary Challenge in Editing.** Visual/Layout errors (green segments) form the second-largest block in Edit tasks. Closed-source models notably struggle with *Layout Structure* (E3.1) and *Visual Fidelity Gap* (E3.6), indicating that while they can write the logical JavaScript, aligning CSS properties precisely with the user’s aesthetic intent remains difficult.
- **Repair Tasks Fail Primarily due to Unaddressed Defects (E5.1).** Figure 20 reveals a completely different error paradigm for Repair tasks. The newly introduced repair-specific categories (pink/magenta) overwhelmingly dominate the distribution. Specifically, *Defect Not Addressed* (E5.1) is the primary failure mode. For weaker models (e.g., Qwen3-VL-30B-A3B-Thinking), a staggering 74% of errors occur because the generated patch simply fails to fix the original bug. Even Gemini-3-Pro-Preview, the absolute best performer in Repair (§4.2), sees 49% of its errors coming from E5.1.
- **The “Over-editing” Penalty in Repair (E5.3).** We also observe a notable proportion of *New Defect Introduced* (E5.3) errors in Repair tasks (ranging from 8% to 12% for closed-source models). This quantitatively corroborates the heavy right-tail distribution observed in our Patch Complexity Analysis (§4.3.5): models that generate excessively large patches (like Claude and GPT-5.2) frequently break previously working functionality while attempting to fix a localized bug.
- **Modality Consistency.** When splitting the analysis between Text Input and Image Input (bottom panels of both figures), the proportional distribution of error categories remains remarkably stable within each model. This suggests that the core weaknesses—failing to implement complete features in Edit, and failing to locate and fix the defect in Repair—are fundamental reasoning bottlenecks rather than modality-specific perceptual failures.

5 Related Work

Our work is closely related to two research threads: (i) **code-capable foundation models and agentic coding systems**, and (ii) **benchmarks and evaluation frameworks for web development** that require judging both visual quality and interactive correctness.

Code LLMs and code agents. Large language models for code generation have progressed rapidly from early program synthesis benchmarks (Austin et al., 2021; Chen et al., 2021) and competition-level reasoning (Li et al., 2022) to fully interactive coding agents capable of autonomous software engineering. On the model side, both proprietary systems— Gemini-3-Pro (Gemini Team and Google, 2023), Claude-Opus-4.5 (Anthropic, 2025)—and open-source alternatives— Qwen3-Coder (Yang et al., 2025) and OpenCoder (Huang et al., 2024)—have demonstrated strong performance on standard code generation benchmarks such as HumanEval (Chen et al., 2021) and LiveCodeBench (Jain et al., 2024). On the agent side, SWE-agent (Yang et al., 2024b) and OpenHands (Wang et al., 2024) equip LLMs with tool-use interfaces for repository-level software engineering, while commercial platforms such as Devin (Cognition AI, 2024) and Cursor demonstrate the practical viability of agentic coding workflows. SWE-bench (Jimenez et al., 2023) has become the de facto evaluation framework for these agents, driving rapid progress in automated bug fixing and code editing. Nevertheless, web development introduces a distinct challenge compared to algorithmic programming or repository-level repair: success is ultimately reflected in the *user-facing artifact*—layout fidelity, design aesthetics, responsiveness, interaction logic, state transitions, and accessibility. These criteria are difficult to capture with purely code-based metrics and can be missed by evaluation suites designed primarily for functional correctness.

Benchmarks for web coding. Existing web-coding benchmarks can be categorized along two orthogonal axes: *task type* and *input modality*.

From the task perspective, benchmarks often study:

- **Generation:** producing web pages or mini-apps from requirements. This ranges from early UI-to-code work such as pix2code (Beltramelli, 2017) and Web2Code (Yun et al., 2024), to more recent benchmarks including Design2Code (Si et al., 2024) for screenshot-to-HTML conversion, WebGen-Bench (Lu et al., 2025) for interactive website generation from scratch, DesignBench (Xiao et al., 2025) for MLLM-based front-end code generation, and Web-Bench (Xu et al., 2025) for evaluating code against web standards and frameworks. Interaction2Code (Wan et al., 2024) further extends the modality to interactive prototypes, while IWR-Bench (Chen et al., 2025) and FronTalk (Wu et al., 2025) explore video-conditioned and conversational generation settings, respectively.

- **Editing:** modifying an existing codebase to satisfy new requirements. SWE-bench Multimodal (Yang et al., 2024a) extends the original SWE-bench (Jimenez et al., 2023) to visual software domains, requiring models to interpret screenshots alongside issue descriptions.
- **Repair:** fixing defects in UI/UX or broken interactions, ranging from text-described bugs to visually grounded defect descriptions.

Several recent efforts aim at holistic, multi-dimensional coverage. WebUIBench (Lin et al., 2025) benchmarks WebUI-to-code generation with comprehensive metrics; FullFront (Sun et al., 2025) spans the full front-end engineering workflow; ArtifactsBench (Zhang et al., 2025) bridges the visual-interactive gap in LLM code generation evaluation; and WebDev Arena (LMSYS Org, 2024) provides a human-preference-based leaderboard for web development. WebCoderBench (Liu et al., 2026) proposes comprehensive and interpretable evaluation metrics, while WebMMU (Awal et al., 2025) extends coverage to multilingual website understanding.

Despite this growing body of work, most existing benchmarks focus on a single task type (typically generation) or a single input modality (typically text or static images), and their evaluations often rely on either weak proxies (e.g., single screenshot similarity or DOM heuristics) or brittle scripted tests that require strict attribute conventions. WebCompass addresses these limitations by spanning three modalities, three task types, and employing execution-grounded evaluation that tests end-to-end runtime behavior.

Evaluation paradigms for interactive visual artifacts. Evaluation methods for web-facing artifacts commonly fall into three classes:

- **Rule-/test-based evaluation.** Deterministic test suites, as employed by SWE-bench (Jimenez et al., 2023) and Web-Bench (Xu et al., 2025), provide precise and reproducible verdicts but typically require heavy instrumentation, strict naming conventions, and substantial engineering effort to achieve good coverage across diverse implementations.
- **Agent-based interaction.** Web agents—as pioneered by WebArena (Zhou et al., 2023) and extended to multimodal settings in VisualWebArena (Koh et al., 2024)—can explore an artifact by interacting with the page and checking outcomes. However, coverage remains challenging: predefined action spaces may miss complex behaviors, and long-horizon workflows are hard to validate end to end.
- **LLM/MLLM-as-a-Judge.** Language or multimodal referees (Zheng et al., 2023; Ge et al., 2023) can scale to open-ended designs and assess multiple dimensions jointly, but may be subjective without careful rubric design and evidence grounding.

Our work adopts a task-aware combination of these paradigms. For **editing** and **repair**, we use *checklist-guided* LLM-as-a-Judge (Zheng et al., 2023) to anchor evaluation in per-task, fine-grained criteria with evidence grounding. For **generation**, where acceptable solutions are diverse and interactivity is open-ended, we introduce an *Agent-as-a-Judge* protocol (Zhuge et al., 2024) that combines browser-based interaction (via the Model Context Protocol) with iterative test-case synthesis, providing stronger and more realistic validation than any single evaluation paradigm alone.

6 Conclusion

We presented **WebCompass**, a multimodal benchmark unifying generation, editing, and repair across text, image, and video modalities, with task-aware evaluation combining LLM-as-a-Judge and Agent-as-a-Judge protocols to assess executability, functional behavior, and visual quality across task-specific rubrics. Our experiments reveal that closed-source models lead by ~ 25 points over the best open-source alternatives, visual quality remains the most persistent bottleneck even for frontier models, and generation, editing, and repair stress fundamentally different capabilities—no single model dominates all three. These findings suggest that advancing web coding agents requires not only stronger functional reasoning but also deeper visual design understanding and greater output consistency, pointing toward a future where coding agents are evaluated—and optimized—as holistic builders of user-facing experiences rather than mere code generators.

References

Yuxuan Wan, Jingyu Xiao, Man Ho Lam, Junliang Liu, Yintong Huo, and Michael R. Lyu. Interaction2code: Benchmarking mllm-based interactive webpage code generation from interactive prototyping. *arXiv preprint arXiv:2411.03292*, 2024.

-
- Xueqing Wu, Zihan Xue, Da Yin, Shuyan Zhou, Kai-Wei Chang, Nanyun Peng, and Yeming Wen. Frontalk: Benchmarking front-end development as conversational code generation with multi-modal feedback. *arXiv preprint arXiv:2601.04203*, 2025.
- Hongda Zhu, Yiwen Zhang, Bing Zhao, Jingzhe Ding, Siyao Liu, Tong Liu, Dandan Wang, Yanan Liu, and Zhaojian Li. Frontendbench: A benchmark for evaluating llms on front-end development via automatic evaluation. *arXiv preprint arXiv:2506.13832*, 2025.
- Zimu Lu, Yunqiao Yang, Houxing Ren, Haotian Hou, Han Xiao, Ke Wang, Weikang Shi, Aojun Zhou, Mingjie Zhan, and Hongsheng Li. Webgen-bench: Evaluating llms on generating interactive and functional websites from scratch. *arXiv preprint arXiv:2505.03733*, 2025.
- John Yang, Carlos E Jimenez, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024a.
- Jingyu Xiao, Ming Wang, Man Ho Lam, Yuxuan Wan, Junliang Liu, Yintong Huo, and Michael R. Lyu. Designbench: A comprehensive benchmark for mllm-based front-end code generation. *arXiv preprint arXiv:2506.06251*, 2025.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Liber, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024b.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, et al. Openhands: An open platform for ai software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Cognition AI. Introducing devin, the first ai software engineer. <https://www.cognition.ai/blog/introducing-devin>, 2024. Accessed: 2025-01-15.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.
- Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, Wenyi Wang, Dmitrii Khizbullin, Yunyang Xiong, Zechun Liu, Ernie Chang, Raghuraman Krishnamoorthi, Yuandong Tian, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.
- Chenchen Zhang, Yuhang Li, Can Xu, Jiaheng Liu, Ao Liu, Changzhi Zhou, Ken Deng, Dengpeng Wu, Guanhua Huang, Kejiao Li, et al. Artifactsbench: Bridging the visual-interactive gap in llm code generation evaluation. *arXiv preprint arXiv:2507.04952*, 2025.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.
- Gemini Team and Google. Gemini: A family of highly capable multimodal models, 2023.
- Anthropic. Claude. <https://claude.ai/>, 2025. Accessed: 2026-03-15.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

-
- Siming Huang, Tianhao Cheng, Jason Klein Liu, Jiaran Hao, Liuyihan Song, Yang Xu, J Yang, JH Liu, Chenchen Zhang, Linzheng Chai, et al. Opencoder: The open cookbook for top-tier code large language models. *arXiv preprint arXiv:2411.04905*, 2024.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.
- Tony Beltramelli. pix2code: Generating code from a graphical user interface screenshot. *arXiv preprint arXiv:1705.07962*, 2017.
- Sukmin Yun, Haokun Lin, Rusiru Thushara, Mohammad Qazim Bhat, Yongxin Wang, Zutao Jiang, Mingkai Deng, Jinhong Wang, Tianhua Tao, Junbo Li, et al. Web2code: A large-scale webpage-to-code dataset and evaluation framework for multimodal llms. *arXiv preprint arXiv:2406.20098*, 2024.
- Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. Design2code: Benchmarking multimodal code generation for automated front-end engineering. *arXiv preprint arXiv:2403.03163*, 2024.
- Kai Xu, YiWei Mao, XinYi Guan, and ZiLong Feng. Web-bench: A llm code benchmark based on web standards and frameworks. *arXiv preprint arXiv:2505.07473*, 2025.
- Yang Chen, Minghao Liu, Yufan Shen, Yunwen Li, Tianyuan Huang, Xinyu Fang, Tianyu Zheng, Wenxuan Huang, Cheng Yang, Daocheng Fu, et al. Iwr-bench: Can llms reconstruct interactive webpage from a user interaction video? *arXiv preprint arXiv:2509.24709*, 2025.
- Zhiyu Lin, Zhengda Zhou, Zhiyuan Zhao, Tianrui Wan, Yilun Ma, Junyu Gao, and Xuelong Li. Webuibench: a comprehensive benchmark for evaluating multimodal large language models in webui-to-code. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 15780–15797, 2025.
- Haoyu Sun, Huichen Will Wang, Jiawei Gu, Linjie Li, and Yu Cheng. Fullfront: Benchmarking mllms across the full front-end engineering workflow. *arXiv preprint arXiv:2505.17399*, 2025.
- LMSYS Org. WebDev Arena: Benchmarking LLMs on Web Development. <https://web.lmarena.ai/leaderboard/webdev>, 2024. Accessed: 2025-01-15.
- Chenxu Liu, Yingjie Fu, Wei Yang, Ying Zhang, and Tao Xie. Webcoderbench: Benchmarking web application generation with comprehensive and interpretable evaluation metrics. *arXiv preprint arXiv:2601.02430*, 2026.
- Rabiul Awal, Mahsa Massoud, Aarash Feizi, Zichao Li, Suyuchen Wang, Christopher Pal, Aishwarya Agrawal, David Vazquez, Siva Reddy, Juan A Rodriguez, et al. Webmmu: A benchmark for multimodal multilingual website understanding and code generation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 25129–25156, 2025.
- Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, 2024.
- Wentao Ge, Shunian Chen, Guiming Hardy Chen, Junying Chen, Zhihong Chen, Nuo Chen, Wenya Xie, Shuo Yan, Chenghao Zhu, Ziyue Lin, et al. Mllm-bench: evaluating multimodal llms with per-sample criteria. *arXiv preprint arXiv:2311.13951*, 2023.
- Google. Gemini. <https://gemini.google.com/>, 2025. Accessed: 2026-03-15.
- OpenAI. Openai. <https://openai.com/>, 2025. Accessed: 2026-03-15.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-v1 technical report. *arXiv preprint arXiv:2511.21631*, 2025.

A Appendix

A.1 Limitations

While WebCompass represents a substantial step toward comprehensive evaluation of web coding agents, we acknowledge several limitations that future work should address.

Front-end focus. WebCompass concentrates exclusively on front-end web development (HTML, CSS, JavaScript, and front-end frameworks). It does not evaluate back-end capabilities such as database design, server-side logic, API development, or deployment workflows. Real-world web engineering involves full-stack development, and extending the benchmark to cover back-end tasks would provide a more complete assessment.

Structured queries vs. creative intent. For generation tasks, we deliberately refine underspecified user queries into structured web design documents (specifying content, interaction, and visual appearance) to enable reproducible, automated evaluation. This design choice means that our benchmark primarily tests *instruction-following* capability rather than the ability to interpret vague, creative intent. We acknowledge this as an inherent trade-off: WebCompass prioritizes *deterministic evaluation standards over open-ended creative assessment*. Complementary benchmarks that explicitly measure creative divergence (e.g., via human preference ranking) would provide a valuable additional perspective.

Limited real-time interaction with dynamic web pages. Our evaluation protocols currently cannot perform real-time interaction with highly dynamic web pages—such as browser-based games or applications with frequent state transitions—in the way a human would. While our framework supports both natural interactions and script-based inspection, it remains challenging to keep pace with rapidly evolving page states, making it difficult to accurately assess time-sensitive behaviors such as real-time game logic, continuous animation responses, or state transitions that depend on precise timing. As a result, the evaluation quality for such highly dynamic web pages may not fully reflect their actual functionality and user experience.

Static benchmark and contamination risk. As a static benchmark, WebCompass is susceptible to data contamination if future models are trained on data that includes our tasks or similar web pages. While we mitigate this through diverse data sources and original task synthesis, maintaining a contamination-free evaluation over time may require periodic updates or dynamic task generation.

Evaluation cost. The Agent-as-a-Judge protocol, while more thorough than static evaluation, is computationally expensive. Each generation task requires launching a headless browser, executing multi-step interaction sequences, and synthesizing iterative test cases, which significantly increases evaluation time and cost compared to simpler metrics. This may limit the benchmark’s accessibility for resource-constrained research groups.

A.2 Disclosure of LLM Assistance

The authors independently conceived and executed all scientific ideas, algorithmic implementations, and experimental data analyses. Large Language Models (LLMs) were employed exclusively as auxiliary tools for language editing and enhancing the clarity of the manuscript. No experimental data, training samples, or reported results were generated using LLMs without rigorous human verification.

A.3 Per-Dimension Framework Evaluation

Table 6 provides the per-dimension breakdown of the framework subset evaluation summarized in Section 4.3.2. Dimension abbreviations follow Table 3.

A.4 Model Card

Table 7 lists all model variants referenced in the experiments, including the auxiliary comparison model used in Section 4.3.7.

Table 6: Per-dimension subset evaluation across different front-end frameworks. Each model is tested on 60 randomly sampled tasks per category using React, Vue, and Vanilla HTML/JS. **Green bold**: best; **blue underline**: second best per framework.

Model	FW	Generation			Edit			Repair		
		RUN.	SPI.	DSQ.	ITG.	FTI.	STC.	RCT.	ITI.	RFE.
GPT-5.2	React	62.08	60.57	<u>47.88</u>	43.60	40.10	35.86	54.58	82.40	62.56
	Vue	<u>65.08</u>	<u>56.79</u>	45.29	<u>45.73</u>	<u>43.30</u>	<u>38.38</u>	<u>48.85</u>	76.77	59.14
	Vanilla	75.13	60.18	56.38	65.20	61.87	55.79	44.10	<u>79.82</u>	<u>60.34</u>
Gemini-3-Pro-Preview	React	61.05	47.29	46.11	<u>54.37</u>	<u>50.31</u>	<u>43.92</u>	<u>44.02</u>	<u>81.67</u>	<u>64.12</u>
	Vue	<u>71.01</u>	55.98	<u>59.61</u>	43.58	39.13	34.23	39.25	74.70	57.11
	Vanilla	75.02	<u>55.70</u>	64.49	74.84	69.48	62.06	50.95	86.62	65.78
Claude-Opus-4.5	React	<u>79.16</u>	71.91	56.78	<u>55.84</u>	<u>49.10</u>	<u>42.24</u>	<u>44.47</u>	78.37	<u>63.97</u>
	Vue	72.85	66.99	<u>58.74</u>	40.14	38.27	30.04	41.14	<u>81.94</u>	62.85
	Vanilla	79.23	<u>71.72</u>	62.59	78.31	72.47	65.37	47.21	85.56	64.38
Qwen3-VL-235B-A22B-Instruct	React	<u>45.76</u>	<u>25.80</u>	<u>36.11</u>	<u>20.57</u>	<u>18.15</u>	<u>15.84</u>	26.16	<u>62.63</u>	<u>45.33</u>
	Vue	42.18	23.17	27.46	15.12	16.02	13.31	22.22	57.00	37.56
	Vanilla	62.16	39.97	45.01	26.33	24.99	22.19	<u>25.09</u>	71.46	46.89

Table 7: List of model variants referenced in the experiments.

Model
Claude-Opus-4.5 (Anthropic, 2025)
Claude-Sonnet-4.5 (Anthropic, 2025)
Gemini-3-Pro-Preview (Google, 2025)
Gemini-3-Flash-Preview (Google, 2025)
GPT-5.2 (OpenAI, 2025)
Qwen3-32B (Yang et al., 2025)
Qwen3-VL-32B-Instruct (Bai et al., 2025)
Qwen3-VL-235B-A22B-Instruct (Bai et al., 2025)
Qwen3-VL-235B-A22B-Thinking (Bai et al., 2025)
Qwen3-VL-30B-A3B-Instruct (Bai et al., 2025)
Qwen3-VL-30B-A3B-Thinking (Bai et al., 2025)

A.5 Detailed Worst-of- n Stability Results

Table 8 extends the Worst-of- n stability analysis presented in the main text by reporting $Pass@1$, $W@2$, and $W@4$ scores across all nine evaluation dimensions, grouped by task category.

A.6 Prompt Templates

This section presents the prompt templates used in our pipeline, covering task prompts for code generation, editing, and repair (§A.6.3–A.6.5), evaluation prompts for LLM-as-a-Judge and Agent-as-a-Judge (§A.6.6–A.6.7), and auxiliary prompts for checklist generation and error analysis (§A.6.2–A.6.1).

A.6.1 Error Analysis Prompt

Below is the prompt used to classify point deductions into standardized error types and root causes. The full prompt includes the complete taxonomy definitions, a decision flowchart, point allocation rules, and worked examples.

Error Analysis Prompt (Part 1: Taxonomy & Rules)

You are a web development QA analyst. Classify each point deduction from LLM-generated web pages into standardized error types.

== Error Type Taxonomy (3-level) ==

E1 - Code Execution Errors

E1.1 | Page Crash / White Screen | Page fails to render entirely

E1.2 | Console Error (Blocking) | JS error that prevents feature from working

Table 8: Consistency and Stability Results for Gemini-3-Pro-Preview and Qwen3-VL-235B-A22B-Instruct ($n = 4$). $Pass@1$, $W@2$, and $W@4$ are reported across all nine evaluation dimensions grouped by task category; $\Delta\downarrow$ denotes the relative drop from $Pass@1$ to $W@4$.

Metric	Generation			Editing			Repair		
	RUN.	SPI.	DSQ.	ITG.	FTI.	STC.	RCT.	ITI.	RFE.
<i>Gemini-3-Pro-Preview</i>									
Pass@1 (%)	75.31	56.90	60.68	74.84	69.48	62.06	50.95	86.62	65.78
W@2 (%)	73.55	47.74	56.03	67.41	62.41	53.77	39.83	80.41	58.19
W@4 (%)	68.92	39.80	49.27	63.45	57.89	49.51	31.31	72.07	49.80
$\Delta\downarrow$ (%)	8.48	30.05	18.80	15.22	16.68	20.22	38.55	16.80	24.29
<i>Qwen3-VL-235B-A22B-Instruct</i>									
Pass@1 (%)	61.22	38.59	42.77	26.33	24.99	22.19	25.09	71.46	46.89
W@2 (%)	52.75	28.42	34.57	19.54	17.62	17.64	21.00	62.46	38.68
W@4 (%)	44.42	23.56	29.09	15.91	14.50	13.76	17.66	58.23	32.86
$\Delta\downarrow$ (%)	27.44	38.95	31.99	39.57	41.98	37.99	29.61	18.51	29.92

E1.3 | Resource Loading Failure | Missing images, fonts, external dependencies

E2 - Functional Errors

- E2.1 | Feature Missing | A required feature/component is entirely absent
- E2.2 | Feature Incomplete | Feature exists but is only partially implemented
- E2.3 | Logic Error | Feature exists but produces wrong behavior/output
- E2.4 | Interaction Error | Wrong event type, trigger condition, or user flow
- E2.5 | Data/Content Error | Wrong text, numbers, items, or dynamic content

E3 - Visual/Style Errors

- E3.1 | Layout Structure Error | Wrong arrangement, grid, flex direction, positioning
- E3.2 | Color/Theme Error | Wrong colors, gradients, or visual theme
- E3.3 | Typography Error | Wrong font, size, weight, line-height
- E3.4 | Spacing/Sizing Error | Wrong margin, padding, width, height
- E3.5 | Animation/Transition Error | Missing, wrong, or poorly executed animations
- E3.6 | Visual Fidelity Gap | General mismatch with reference image/video

E4 - Non-Functional Errors

- E4.1 | Performance Issue | Lag, high CPU, memory leak, slow rendering
- E4.2 | Accessibility Issue | Missing alt text, poor contrast, no keyboard nav
- E4.3 | Responsiveness Issue | Breaks at different viewport sizes

== Root Cause Analysis (select the MOST specific one) ==

- Requirement Misunderstanding | Prompt clearly stated X, LLM built Y
- Requirement Omission | LLM skipped/forgot an explicit requirement
- Insufficient Reproduction | Visual/behavioral gap vs reference image/video
- Capability Limitation | LLM lacks skill for this technique
- Hallucination | LLM used non-existent APIs or fabricated behavior
- Oversimplification | LLM took shortcuts, used simpler approach
- null | Cannot determine or none of the above

== Decision Flowchart ==

For each deducted point:

1. Does the page crash or show blocking console errors? -> E1.x
2. Does a required feature not work as specified? -> E2.x
3. Does it work but look wrong? -> E3.x
4. Does it look right but have non-functional issues? -> E4.x

== Points Allocation Rule ==

When a single checklist item has multiple issues mentioned in reason:

- Allocate points proportionally to issue severity
- If unclear, split evenly among identified issues
- Critical failures get more points than minor issues

== Output Format ==

Return a JSON array:

```
[{"checklist_id": <id>, "task": "<task>",
  "score": <score>, "max_score": <max_score>,
```

```
"errors": [{"type": "<E1.1|...|E4.3>",
  "description": "<concise description>",
  "points_deducted": <number>,
  "root_cause": "<label|null>"}]]
```

== Rules ==

1. sum(points_deducted) must equal max_score - score for each item
2. Each error gets exactly ONE type code
3. Full-score items -> empty errors array
4. Multiple distinct issues -> separate error objects

Error Analysis Prompt (Part 2: Few-Shot Examples)

== Example 1: Runtime error ==

Input: {"id":1, "task":"Page loads correctly",
"max_score":5, "score":0,
"reason":"Uncaught ReferenceError: initApp is not defined.
Page shows white screen."}

Output:

```
[{"checklist_id":1, "task":"Page loads correctly", "score":0,
  "max_score":5, "errors":[{"type":"E1.2",
    "description":"Uncaught ReferenceError prevents page init",
    "points_deducted":5, "root_cause":"Hallucination"}]]
```

== Example 2: Multiple issues in one item ==

Input: {"id":3, "task":"Test mouse wake effect",
"max_score":12, "score":6,
"reason":"Wake works on mousedown+drag but NOT on
hover-only movement. Dissipation too slow (visible after 3s)."}
Output:

```
[{"checklist_id":3, "task":"Test mouse wake effect", "score":6,
  "max_score":12, "errors":[
    {"type":"E2.4",
      "description":"Wake requires mousedown instead of hover",
      "points_deducted":4,
      "root_cause":"Requirement Misunderstanding"},
    {"type":"E2.3",
      "description":"Wake dissipation too slow (>3s)",
      "points_deducted":2, "root_cause":null}]]
```

== Example 3: Style issue ==

Input: {"id":5, "task":"Check header layout",
"max_score":6, "score":4,
"reason":"Logo and nav are present but laid out vertically
instead of horizontal row."}

Output:

```
[{"checklist_id":5, "task":"Check header layout", "score":4,
  "max_score":6, "errors":[{"type":"E3.1",
    "description":"Header elements stacked vertically  
instead of required horizontal layout",
    "points_deducted":2,
    "root_cause":"Insufficient Reproduction"}]]
```

Now analyze the following checklist:

[CHECKLIST]

A.6.2 Checklist Generation Prompt

Below is the prompt used to generate evaluation checklists. The prompt instructs an LLM to produce structured checklist items spanning three dimensions: Runnability, Spec Implementation, and Design Quality. We show the core instructions and an abridged example; the full prompt includes a complete 13-item few-shot example.

Checklist Generation Prompt (Part 1: Role & Format)

You are a senior and extremely detail-oriented code review expert. You are proficient in multiple programming languages, frontend technologies, interaction design, and UI aesthetics. Your task is to generate a checklist for evaluating responses to the given [query].

Role Definition

- Responsibility: Act as a member of an authoritative technical review committee--objective, comprehensive, and impartial.
- Attitude: Meticulous, professional, and uncompromising.
- Aesthetic Standards: Excellent design taste and high standards for user experience.

Output Format Requirements (Each checklist item must include)

- task: A clear, single-purpose task for the UI agent to verify.
- category: One of: Runnability | Spec Implementation | Design Quality
- operation_sequence: Steps the UI agent should perform (2-4

-
- steps, verifiable through screenshots/screen recordings).
 - expected_result: Specific, observable success criteria.
 - criteria: Strict scoring rules (clear pass/fail thresholds).
 - max_score: Maximum score for this item (must be a number).

Checklist Generation Prompt (Part 2: Dimensions)

```
## Evaluation Dimensions and Checklist Structure

### 1. Runnability (Fixed 1 item, worth 10 points)
Fixed universal checklist item:
{"task": "Does the page load correctly and run without errors?",
 "category": "Runnability",
 "operation_sequence": "1. Open Console panel 2. Load the page
 3. Check for red error messages 4. Check Network for 404/500",
 "expected_result": "Page loads completely, no Console errors,
  no failed requests, all static resources load successfully",
 "criteria": "Full 10 pts; JS errors -5; Resource 404 -3;
  White screen = 0; Warnings do not deduct",
 "max_score": 10}

### 2. Spec Implementation (6-10 items, worth 60-70 points)
Core part -- generated entirely based on the Query:
- First 4-5 items: core functionality, main interactions,
  key user journeys (most stringent, hardest to pass)
- Remaining items: secondary features, edge cases, robustness
- Score per item: 8-15 points, allocated by importance

### 3. Design Quality (1 universal + 1-2 Query-specific, 20-25 pts)
Universal item evaluates: color harmony, layout spacing (8px
multiple principle), typography (body font >=14px, line-height
>1.5x). Query-specific items check domain visuals (e.g., game
board design, chart readability, animation smoothness).

## Important Notes
- Hard requirement: sum of all max_score must equal 100
- Hard requirement: 10-16 checklist items total
- Each item must be verifiable through screenshots/console/
  network panel/visual alignment checks
```

Checklist Generation Prompt (Part 3: Abridged Example)

```
## Complete Example (abridged -- showing 3 of 13 items)
Query: [Multiplayer online chess game with lobby, game board,
  result modal, and profile pages...]

[{"task": "Does the page load correctly and run without errors?",
 "category": "Runnability",
 "operation_sequence": "1. Open Console 2. Load page ...",
 "expected_result": "Page loads, no errors, all resources OK",
 "criteria": "Full 10; JS errors -5; 404 -3; crash = 0",
 "max_score": 10},
 {"task": "Test chessboard piece selection and valid move
  indicators functionality",
 "category": "Spec Implementation",
 "operation_sequence": "1. Start game 2. Click white pawn (e2)
 3. Verify highlights and move indicators 4. Click valid
  square to confirm move with animation",
 "expected_result": "Piece highlighted, dots on empty squares,
  rings on captures, smooth ~200ms animation",
 "criteria": "Full 12; no highlight -4; no indicators -4;
  no animation -2; wrong move -2",
 "max_score": 12},
 {"task": "Verify dark theme with correct primary/accent colors",
 "category": "Design Quality",
 "operation_sequence": "1. Screenshot Lobby and Game pages
 2. Verify dark gradient background 3. Check primary/accent
  colors 4. Verify semi-transparent cards",
 "expected_result": "Dark gradient bg, wood-brown primary,
  royal blue interactive, gold accents, cohesive dark theme",
 "criteria": "Full 5; light theme -5; inconsistent colors -2",
 "max_score": 5},
  ... (10 more items omitted for brevity)]

Query:
---
[QUERY]
---
```

A.6.3 Generation Prompts

We use three generation prompts corresponding to the three input modalities: text, image, and video. All share a common output contract requiring pure Markdown with fenced code blocks. Below we present

each variant.

Text-Guided Generation Prompt

You are a highly skilled professional front-end engineer.

Your task: Based on the web design document below, generate a complete runnable web project repository.

Hard output contract (MUST follow):

- 1) Your entire response MUST be pure Markdown text.
- 2) ABSOLUTELY NO explanations, no extra commentary.
- 3) Every file MUST be emitted using the following format:
path/to/file.ext
```ext  
<full file content>  
```
- 4) The heading line MUST start with '# ' followed by the file path (relative path).
- 5) The code fence language MUST match the file type.
- 6) Include all necessary files so the project can run.
- 7) Do NOT nest triple backticks inside code blocks.

Few-shot examples:

```
# index.html
```html
<!doctype html>
<html>
 <head>
 <meta charset="utf-8" />
 <title>Demo</title>
 <link rel="stylesheet" href="styles.css" />
 </head>
 <body>
 Hello
 <script type="module" src="main.js"></script>
 </body>
</html>
```

# styles.css
```css
body { font-family: system-ui; }
```

# main.js
```js
console.log('ok')
```
```

Web design document:

```
---
[DOCUMENT]
---
```

Vision-Guided Generation Prompt

You are a highly skilled professional front-end engineer.

Your task: Based on the web design document and the reference screenshots provided, generate a complete runnable web project repository. The screenshots represent the target UI and function. The generated site should match the screenshots as closely as possible.

Hard output contract (MUST follow):

1-7) [Same output contract as Text-Guided Generation]

Startup requirements:

- MUST include a README.md with the simplest way to run locally.
- MUST be runnable via a static server (no backend).
- Prefer Vite or plain static files.

Web design document:

```
---
[DOCUMENT]
---
```

Video-Guided Generation Prompt (Part 1: Analysis Protocol)

You are a world-class front-end engineer with expertise in creating pixel-perfect web reproductions. Your task is to analyze these video frames and create a complete, production-ready web

project repository that exactly replicates the demonstrated interface with professional-grade quality.

MISSION: Create a flawless HTML reproduction that matches the video demonstration in every detail, achieving 95%+ quality score.

COMPREHENSIVE ANALYSIS PROTOCOL:

1. TEMPORAL SEQUENCE ANALYSIS:
 - Study frame progression to understand user interactions
 - Identify animation sequences, timing, and easing patterns
 - Map state transitions and user feedback mechanisms
 - Recognize loading states, hover effects, micro-interactions
 - Document exact timing and duration of animations
2. VISUAL DESIGN EXTRACTION:
 - Extract precise color values (prefer hex codes: #RRGGBB)
 - Identify typography: families, sizes, weights, line heights
 - Measure spacing: margins, padding, gaps (use rem/em units)
 - Analyze shadows: box-shadow values, blur, spread, inset
 - Document border radius, opacity, and gradient effects
 - Note z-index layering and stacking contexts
3. LAYOUT & STRUCTURE ANALYSIS:
 - Identify layout systems: Flexbox, CSS Grid, or positioning
 - Map responsive breakpoints and mobile adaptations
 - Document component hierarchy and nesting structure
 - Analyze alignment, distribution, and spacing patterns
4. INTERACTION PATTERN RECOGNITION:
 - Button states: normal, hover, active, focus, disabled
 - Animation triggers: click, hover, scroll, load events
 - State management: data flow and component updates
 - User feedback: visual confirmations and error states

Video-Guided Generation Prompt (Part 2: Implementation)

TECHNICAL IMPLEMENTATION REQUIREMENTS:

HTML5 STRUCTURE (MANDATORY):

- Use semantic HTML5 elements (main, section, article, nav)
- Include proper meta tags: charset, viewport, description
- Implement accessibility: ARIA labels, alt text, roles
- Ensure valid HTML5 markup

CSS IMPLEMENTATION (MANDATORY):

- Use CSS custom properties (variables) for theming
- Implement modern layout: Flexbox and CSS Grid
- Create smooth animations with cubic-bezier easing
- Use transform3d for hardware acceleration
- Implement responsive design with mobile-first approach
- Include CSS reset/normalize for consistency

JAVASCRIPT FUNCTIONALITY (MANDATORY):

- Write modern ES6+ with proper error handling (try/catch)
- Use requestAnimationFrame for smooth animations
- Add performance optimizations: debouncing, throttling
- Implement proper event delegation and cleanup

ANIMATION & INTERACTION STANDARDS (MANDATORY):

- Use CSS transforms for performance (translate3d, scale)
- Implement 60fps smooth animations
- Add appropriate transition durations (200-500ms)
- Create natural easing: ease-out entrance, ease-in exit

OUTPUT SPECIFICATIONS (CRITICAL):

- Your entire response MUST be pure Markdown text.
- ABSOLUTELY NO explanations or extra commentary.
- Every file MUST use: # path/to/file.ext ``ext ... ``
- Keep assets inline; avoid external dependencies.

Begin your comprehensive analysis and create the ultimate reproduction that achieves 95%+ quality score.

A.6.4 Editing Prompts

The text-guided and vision-guided editing tasks share the same system prompt. The only difference is that the vision-guided variant additionally includes current-state screenshots in the user message. We present the shared system prompt once, followed by the two user-message variants.

Editing System Prompt (shared by Text & Vision variants)

You are an expert frontend developer. Your task is to edit the provided web code based on the given instructions. You will receive the current code and a set of editing instructions.

****Output Format Requirements:****

- Use search/replace blocks to indicate modifications
- Each block must be wrapped in `<search_replace path="...">` `</search_replace>` tags
- The `path` attribute must specify the relative file path (e.g., `"index.html"`, `"resources/style.css"`)
- Each block must contain one `<search>` and one `<replace>`

Return XML format with the following structure:

```
<search_replace path="path/to/file">
<search>
exact text to find in the original file
</search>
<replace>
replacement text with the modification applied
</replace>
</search_replace>
```

If you want to create additional files:

```
<search_replace path="path/to/new_file">
<search></search>
<replace>
complete code for the new file
</replace>
</search_replace>
The search block for new files should be empty.
```

Important:

- The `<search>` block must contain the EXACT text from the original file (including whitespace and indentation).
- The `<replace>` block contains the modified code.
- One `<search_replace>` block can only contain one pair of `<search>` and `<replace>`.
- You can include multiple `<search_replace>` blocks if you need to modify multiple locations or files.
- You must complete the task in single response.

Text-Guided Editing — User Message

```
== User Message ==
## Task Description
Task 0 - {task_type}: {description}
Task 1 - {task_type}: {description}
...

## Source Code
The following is the current code that needs to be modified:
<code_context>
<file path="index.html"> ... </file>
<file path="style.css"> ... </file>
</code_context>
```

Vision-Guided Editing — User Message (extends Text variant)

```
== User Message ==
## Task Description
Task 0 - {task_type}: {description}
...

## Source Code
<code_context>
<file path="index.html"> ... </file>
</code_context>

## Current State Screenshots
The following screenshots show the current state:
[Current Screenshot: page_1.png]
[image_1]
[Current Screenshot: page_2.png]
[image_2]
...
```

A.6.5 Repair Prompts

Similarly, the diagnostic and visual-diagnostic repair tasks share the same system prompt. The visual-diagnostic variant additionally includes before-fix and target-state screenshots.

Repair System Prompt (shared by Diagnostic & Visual variants)

You are an expert frontend developer. Your task is to repair the provided web code based on the given defect types. You will receive the current code with a set of defect types to fix.

Here are the issue types and explanations (the code may not contain all of these issues):

- Occlusion: Elements incorrectly layered causing important content to be covered due to improper z-index or positioning.
- Crowding: Elements too close together due to missing or insufficient spacing (margins/padding).
- Text Overlap: Text overlaps with other elements due to insufficient container size or improper positioning.
- Alignment: Elements not properly aligned with the grid system or sibling elements.
- Color Contrast: Insufficient contrast between text and background colors affecting readability.
- Overflow: Content exceeds container boundaries without proper overflow handling.
- Sizing Proportion: Elements have incorrect dimensions or aspect ratios that distort their appearance.
- Loss of Interactivity: Interactive elements disabled or blocked via disabled attributes or pointer-events: none.
- Semantic Error: Semantic HTML tags replaced with generic divs or spans reducing accessibility.
- Nesting Error: HTML elements nested in invalid ways that violate HTML specifications.
- Missing Attributes: Required attributes missing from elements (e.g., alt, aria-label).

****Output Format Requirements:****

[Same search/replace format as Editing prompts]

Important:

[Same constraints as Editing prompts]

Diagnostic Repair — User Message

== User Message ==

You have only {N} issues to fix, and you cannot fix more than {N} issues.

Source Code

The following is the current code that needs to be modified:

```
<code_context>
<file path="index.html"> ... </file>
</code_context>
```

Visual-Diagnostic Repair — User Message (extends Diagnostic)

== User Message ==

You have only {N} issues to fix, and you cannot fix more than {N} issues.

Source Code

```
<code_context>
<file path="index.html"> ... </file>
</code_context>
```

Current State Screenshots

The following screenshots show the current (defective) state:

[Current Screenshot: page_1.png]

[image_1]

...

Target State Screenshots

The following screenshots show the expected result:

[Target Screenshot: page_1.png]

[image_1]

...

A.6.6 LLM-as-a-Judge Prompts

We use separate judge prompts for editing and repair tasks, each with task-specific scoring dimensions (0–10 scale). The repair judge additionally receives ground-truth code modifications and fixed screenshots as reference.

Edit Task Judge Prompt

```
## Task Description
You are evaluating whether code modifications properly implement
user's instructions for UI changes. You will receive:
1. Task Instructions: multi-line text, each line follows:
    Task <idx> - <task_type>: <description>
2. Generated Code Modifications: the search/replace blocks
3. Original UI Screenshot: the before-modification state
4. Modified UI Screenshot: the after-modification visual result

## Evaluation Framework
Score each task independently across three dimensions (0-10):
- Instruction Targeting: Patch applicability and task-attempt coverage
- Feature Integrity: Whether original and new functionality is correct
- Style Conformance: Visual quality and consistency with original style

## Evaluation Criteria
### 1. Instruction Targeting (0-10 points)
- Are the search/replace blocks syntactically correct?
- Does the modified code run without errors?
- Are changes applied in the correct files/locations?

### 2. Feature Integrity (0-10 points)
- Are original UI interactions preserved?
- Do newly added components provide the required interactivity?
- Are there regressions in functionality?

### 3. Style Conformance (0-10 points)
- Does the modified UI match expected visual outcome?
- Is the visual style consistent where not changed?
- Are colors, fonts, spacing, and layout harmonious?

## Important Notes
- CRITICAL: You MUST evaluate ALL tasks listed
- Visual results are the PRIMARY indicator of success
- Output ONLY valid JSON, no additional text

## Output Format
{"task_scores": [{"task_idx": 0,
  "task_type": "<from Task 0 - <task_type>: ...>",
  "reasoning": "<detailed evaluation>",
  "instruction_targeting": <0-10>,
  "feature_integrity": <0-10>,
  "style_conformance": <0-10>}, ...]}
```

Repair Task Judge Prompt

```
## Task Description
You are evaluating the effectiveness of UI defect repair tasks.
You will receive:
1. Defect Description: multi-line text, each line follows:
    Defect <idx> - <task_type>: <description>
2. Ground-Truth Code Modifications: the ideal fix (reference)
3. Generated Code Modifications: the produced fix
4. Before-Fix UI Screenshot: defective state (red box markers)
5. After-Fix UI Screenshot: the actual repair result
6. Ground-Truth Fixed UI Screenshot: the ideal fix result

## Evaluation Framework
Score each defect repair independently (0-10 per dimension):
- Root-Cause Targeting: Patch applicability and root-cause localization
- Interaction Integrity: Whether original and repaired functionality is correct
- Reference Fidelity: Visual quality vs. ground-truth reference

## Evaluation Criteria
### 1. Root-Cause Targeting (0-10 points)
- Are the search/replace blocks syntactically correct?
- Does the modified code run without errors?
- Do changes target the root cause without introducing errors?

### 2. Interaction Integrity (0-10 points)
- Are original UI interactions preserved after the fix?
- Are repaired elements usable?
- Are there regressions in functionality?

### 3. Reference Fidelity (0-10 points)
- How closely does the fix match the ground-truth reference?
- Are layout, colors, typography aligned with the reference?
- Does the fix look natural and harmonious?

## Important Notes
- CRITICAL: You MUST evaluate ALL defects listed
- Visual comparison with ground-truth is the PRIMARY indicator
- Compare the generated fix with both the defective state and
```

```

    the ideal ground-truth fix
- Output ONLY valid JSON, no additional text

## Output Format
{"task_scores": [{"task_idx": 0,
  "task_type": "<from Defect 0 - <task_type>: ...>",
  "reasoning": "<detailed evaluation>",
  "root_cause_targeting": <0-10>,
  "interaction_integrity": <0-10>,
  "reference_fidelity": <0-10>}, ...]}

```

A.6.7 Agent-as-a-Judge Prompt

The Agent-as-a-Judge system uses two prompt templates: one for generation (producing code from a task description) and one for verification (scoring the generated webpage against a checklist via browser interaction). The verification prompt has two variants—with and without reference images—that share the same execution flow. We present the shared verification prompt below; the image variant additionally instructs the agent to review reference screenshots under the `screenshots/` directory.

Agent-as-a-Judge: Generation Prompt

```

<task>
{problem_statement}
</task>

Based on the design requirements in <task>, generate a complete
web project repository.

```

Agent-as-a-Judge: Verification Prompt (Part 1: Objective & Rules)

You are a strict QA website tester who must rigorously evaluate the execution, aesthetics, and interactive functionality of a webpage based on a checklist (verification and scoring only -- no fixes).

Below is the user's requirements document for the webpage:
<docs>{instruction}</docs>

Your objective: Based on a real user's browsing journey, use `mcp_tools: mcp__chrome-devtools` to verify whether each item in the checklist is met, update every entry in `checklist.json` where score is null to a definitive score, provide reproducible evidence (reason), and save screenshots to `image/`.

=====

Mandatory Rules (Must Be Followed)

=====

- 1) Absolutely do not modify/fix the original website project code.
- 2) The only content you are allowed to create/modify is:
 - `checklist.json`
 - Screenshot files in the `image/` directory
- 3) Complete all tasks in a single run. Before all tasks are completed, you must call tools for verification in every round.

Agent-as-a-Judge: Verification Prompt (Part 2: Execution Flow)

=====

Mandatory Execution Flow (No Steps May Be Skipped)

=====

Step 0: Prepare Output Directory

- 1) Ensure `image/` folder exists in the project directory.
- 2) Take screenshots for every key state verification.

Step 1: Conduct Code Review First (Read-Only)

- 1) Read repository code related to page entry points, routing, interactions, requests, and error handling.
- 2) Compile verifiable points: entry URLs, key buttons/forms, potential error points, data sources and loading logic.
- 3) Code review only guides test paths; final scores must be based on actual webpage behavior.

Step 2: Read `checklist.json`

- 1) Locate all entries where score is null.
- 2) Extract task / operation_sequence / expected_result.

Step 3: Open and Actually Test the Webpage

- 1) Use `mcp__chrome-devtools` for interactive verification

(clicking, typing, navigating, scrolling, etc.).
2) For each item: perform operations, observe expectations,
take screenshots as evidence. For aesthetics tasks,
combine UI screenshots for scoring.

Step 4: Immediately Write Back to Checklist

1) After each verification, write back to checklist.json:
- score: Change from null to a definitive score.
- reason: Single-line string with reproducible evidence.

=====

Key Rule: Entry Point Failure => Cascading Failure

=====

If the website entry point is unavailable (blank screen/crash/
infinite loading): take screenshot as evidence, assign failure
scores to all dependent items.

=====

Termination Condition

=====

You may only end when no entries with score=null remain in
checklist.json.